ALGORITHMS LABORATORY

Theory and Exercises explained step by step

Author: SamuMGH04

WELCOME

Thanks for choosing these notes. This project is created with the aim to go more deep in C++ programming and explore advanced concepts.

LICENSE

These works are protected and distributed under the license
Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International

- **Attribution:** You must mention the original creator
- **Non Commercial:** You can't sell this material
- **No Derivatives:** You can't modify or change the text.

CLASSI, OGGETTI , OOP

Indice

1 Perché sono necessari Classi e Struct **9**

1.1 Limitazioni della programmazione procedurale 9

1.1.1 Crescita della complessità 9

1.1.2 Maggiore probabilità di errori 10

1.2 La necessità di unire dati e comportamento 10

2 Che cos'è un Oggetto **11**

2.1 Stato e comportamento in un'unica entità 11

2.1.1 Lo stato dell'oggetto 11

2.1.2 Il comportamento dell'oggetto 12

2.2 Unità concettuale: dati e operazioni insieme 12

2.2.1 Parallelismo con il mondo reale 12

3 Dal Dato all'Oggetto **13**

3.1 Raggruppare dati e comportamento in tipi 13

3.2 Vantaggi dell'approccio orientato agli oggetti 14

3.2.1 Miglioramento della leggibilità 14

3.2.2 Riduzione degli errori 14

3.2.3 Modellazione naturale delle entità reali 14

3.3 Definire tipi che rappresentano concetti 15

4 Scrivere le Classi **15**

4.1 Dalle funzioni ai tipi orientati agli oggetti 15

4.2 Che cosa definisce una classe 16

4.3 Rendere privata la rappresentazione ed esporre il comportamento 16

4.3.1 Perché la rappresentazione deve essere privata 16

4.3.2 Perché il comportamento deve essere pubblico 17

5 Classi e Struct **17**
5.1 La differenza pratica tra class e struct . . . 17
5.2 Linee guida stilistiche . . . 18
5.2.1 Quando usare struct . . . 18
5.2.2 Quando usare class . . . 18

6 Anatomia di una Classe **19**
6.1 Un esempio minimale . . . 19
6.2 Output atteso . . . 21
6.3 Spiegazione dettagliata del codice . . . 21
6.3.1 La dichiarazione della classe . . . 21
6.3.2 La sezione pubblica: l'interfaccia . . . 21
6.3.3 La sezione privata: la rappresentazione interna . . . 22
6.4 Osservatori e metodi `const` . . . 22

7 Ogni Oggetto Possiede i Propri Membri Dati **23**
7.1 Oggetti come istanze indipendenti . . . 23
7.2 Esempio: due contatori indipendenti . . . 23
7.3 Output atteso . . . 24
7.4 Spiegazione dettagliata . . . 24

8 Il Nome della Classe è Conosciuto all'Interno della Definizione **25**
8.1 Usare il tipo all'interno della propria dichiarazione . . . 25
8.2 Spiegazione dettagliata del codice . . . 26
8.3 Perché questo è importante . . . 26

9 Incapsulamento **27**
9.1 Separare interfaccia e implementazione . . . 27
9.2 Protezione degli invarianti . . . 27
9.3 Accesso controllato ai dati . . . 28
9.4 Flessibilità nell'implementazione . . . 28
9.5 Miglioramento della leggibilità . . . 28

10 Funzioni Membro **29**
10.1 Funzioni che operano sugli oggetti . . . 29
10.2 Invocazione delle funzioni membro . . . 29
10.3 Esempio con più oggetti . . . 30
10.4 Spiegazione dettagliata . . . 30

11 Il Puntatore `this` **31**
11.1 Come le funzioni membro accedono all'oggetto 31
11.2 Accesso ai membri tramite `this` 31
11.3 Esempio: uso implicito e esplicito di `this` 32
11.4 Spiegazione dettagliata del codice 32
11.5 Perché `this` è importante 32

12 Accesso ai Membri e Precedenza degli Operatori **33**
12.1 Interpretazione delle espressioni 33
12.2 Uso di `this->` per chiarezza 33
12.3 Spiegazione dettagliata 34

13 Funzioni Membro `const` **35**
13.1 Garanzie sullo stato dell'oggetto 35
13.2 Il puntatore `this` in funzioni `const` 35
13.3 Conseguenze pratiche 36
13.4 Esempio illustrativo 36
13.5 Importanza progettuale 36

14 Osservatori e Modificatori **37**
14.1 Ispezionare lo stato: gli osservatori 37
14.2 Modificare lo stato: i modificatori 37
14.3 Perché la distinzione è importante 38

15 Nomenclatura dei Membri **39**
15.1 Convenzione comune: membri dati con suffisso `_` 39
15.2 Perché questa convenzione è utile 39
15.3 Esempio con lista di inizializzazione 39
15.4 Impatto sulla leggibilità e sulla manutenzione 40

16 Dichiarazioni Multiple di Membri Dati **41**
16.1 Una pratica consentita ma sconsigliata 41
16.2 Perché lo stile moderno la scoraggia 41
16.3 Impatto sulla progettazione 42

17 Costruttori **43**
17.1 Creare oggetti validi fin dall'inizio 43
17.2 Cosa accade senza un costruttore 43
17.3 Perché i costruttori sono essenziali 44
17.4 Esempio concettuale 44
17.5 Impatto sulla progettazione 44

18 Liste di Inizializzazione dei Membri **45**
18.1 Inizializzazione prima del corpo del costruttore 45
18.2 Ordine di inizializzazione 45
18.3 Quando la lista di inizializzazione è obbligatoria 46
18.4 Vantaggi progettuali 46

19 Liste di Inizializzazione dei Membri **47**
19.1 Inizializzazione prima del corpo del costruttore 47
19.2 Ordine di inizializzazione 47
19.3 Quando la lista di inizializzazione è obbligatoria 48
19.4 Vantaggi progettuali 48

20 Ordine di Inizializzazione **49**
20.1 L'ordine delle dichiarazioni è determinante 49
20.2 Perché l'ordine della lista non cambia nulla 49
20.3 Rischi di un ordine incoerente 50
20.4 Buona pratica consigliata 50
20.5 Impatto sulla progettazione 51

21 Quando il Costruttore di Default Non Viene Generato **51**
21.1 Regola fondamentale: dichiarare un costruttore sopprime quello di default . 51
21.2 Richiedere esplicitamente il costruttore di default 52
21.3 Costruttori implicitamente generati: riepilogo 52
21.4 Costruttori esplicitamente defaulted 53
21.5 Impatto sulla progettazione 53

22 = default e = delete **54**
22.1 Controllare cosa è permesso 54
22.2 Perché usarli 54
22.3 Esempio: classe non copiabile 55
22.4 Impatto sulla progettazione 55

23 Costruttori Eliminati **56**
23.1 Bloccare operazioni a compile time 56
23.2 Rafforzare gli invarianti della classe 56
23.3 Esempio: classe non copiabile 57
23.4 Impatto sulla progettazione 57

24 Accessors **58**
24.1 Leggere lo stato senza esporlo 58
24.2 Scelta del tipo di ritorno 58

24.3 Esempio di accessor 59
24.4 Ruolo degli accessors nella progettazione 59

25 Overloading degli Operatori **60**
25.1 Stampa su stream con una funzione libera 60
25.2 Esempio completo: un tipo razionale 60
25.3 Centralizzare la logica degli invarianti 61
25.4 Estendere l'esempio: stampa di un `rational` 62
25.5 Impatto sulla progettazione 62

26 Passaggio di Oggetti **63**
26.1 Valore o riferimento: criteri di scelta 63
26.2 Esempi di funzioni con passaggio per riferimento 63
26.3 Impatto sulla progettazione 64

27 Checklist di Progettazione **65**
27.1 Quali dati servono? 65
27.2 Quali operazioni deve offrire la classe? 65
27.3 Quali invarianti devono sempre essere rispettati? 65
27.4 Come garantire oggetti validi al momento della costruzione? 66
27.5 Quali funzioni devono essere `const`? 66

28 Esercizio: progettare una classe `timer` **67**
28.1 Passo 1: dati necessari e invariante 67
28.2 Passo 2: costruzione e avanzamento del timer 67
28.3 Passo 3: stampa del timer 67
28.4 Codice Finale (a blocchi per migliorare estetica ed evitare tagli) 68

29 Perché le funzioni membro sono fondamentali **70**
29.1 La classe come unità di comportamento 70
29.2 Impatto sulla qualità del codice 71
29.3 Decidere cosa appartiene alla classe 71
29.4 Effetti sulla progettazione complessiva 72

30 Funzioni membro e funzioni non–membro **72**
30.1 Due categorie di funzioni 72
30.2 Quando una funzione deve essere membro 72
30.3 Quando una funzione deve restare non–membro 73
30.4 Effetti sulla progettazione complessiva 73

31 Definire le funzioni membro dentro o fuori dalla classe **74**
31.1 Interfaccia e implementazione 74
31.2 Definizione interna alla classe 74
31.3 Definizione esterna alla classe 75
31.4 Vantaggi della separazione 75

32 Dichiarazione e definizione delle funzioni membro **76**
32.1 Il ruolo della dichiarazione 76
32.2 Il ruolo della definizione 76
32.3 Interfaccia vs implementazione 77
32.4 Costruttori: dichiarazione e definizione separate 77

33 Le funzioni membro operano sugli oggetti **78**
33.1 Ogni chiamata riguarda un oggetto preciso 78
33.2 Lo stesso comportamento, stati diversi 78
33.3 Implicazioni progettuali 79

34 Il puntatore implicito `this` **80**
34.1 Che cos'è `this` 80
34.2 Esempio: chiamata a una funzione membro 80
34.3 Come avviene la chiamata 81
34.4 Perché `this` è importante 81
34.5 Un esempio più completo 81

35 Funzioni membro e stato dell'oggetto **82**
35.1 Due categorie di funzioni membro 82
35.2 Perché serve distinguere tra osservatori e modificatori 82
35.3 Il ruolo delle funzioni membro `const` 83
35.4 Esempio completo 83

36 Un vero problema con `const`: funzioni che osservano lo stato **84**
36.1 Funzioni che osservano senza modificare 84
36.2 Quando emergono problemi 84
36.3 Esempio del problema 84
36.4 La soluzione: qualificare correttamente gli osservatori 85
36.5 Perché questo è importante 85

37 Perché `print()` deve essere `const` **86**
37.1 Accesso tramite riferimento costante 86
37.2 La firma corretta di una funzione di stampa 86
37.3 Perché il qualificatore `const` è indispensabile 87

37.4 Effetti sulla progettazione 87

38 Funzioni membro `const` e stato dell'oggetto **88**
38.1 Il problema: tutte le funzioni possono modificare lo stato 88
38.2 La soluzione: funzioni membro `const` 88
38.3 Sintassi corretta 89
38.4 Controllo del compilatore 89
38.5 Impatto sulla progettazione 89

39 Il legame tra `const` e il puntatore implicito `this` **90**
39.1 Come cambia `this` nelle funzioni membro 90
39.2 Perché la modifica è vietata 90
39.3 Esempio illustrativo 91
39.4 Implicazioni progettuali 91

40 Funzioni `const` e non-`const`: un esempio con `point` **92**
40.1 Esempio di classe con osservatori e modificatori 92
40.2 Regola fondamentale: cosa può chiamare un oggetto `const` 92
40.3 Perché questa regola è essenziale 93

41 Livelli di accesso nelle classi **94**
41.1 Interfaccia pubblica e dettagli privati 94
41.2 Come si specifica il livello di accesso 94
41.3 Più livelli di accesso nella stessa classe 95
41.4 Perché i livelli di accesso sono importanti 95

42 Membri `public` e `private` **96**
42.1 Cosa è consentito ai membri `public` 96
42.2 Cosa è consentito ai membri `private` 96
42.3 Il ruolo di `protected` 97
42.4 Implicazioni progettuali 97

43 Implementazione della classe `point`: interfaccia pubblica e rappresentazione privata **98**
43.1 Interfaccia pubblica: costruttori e osservatori 98
43.2 Modificatori e rappresentazione privata 99
43.3 Cosa è cambiato rispetto a una versione ingenua 99
43.4 Errore comune: accesso diretto ai membri privati 100
43.5 Perché questa struttura è corretta 100

44 Interfaccia e implementazione: perché separarli **101**
44.1 L'interfaccia: ciò che gli utenti devono conoscere 101
44.2 L'implementazione: ciò che la classe controlla 101
44.3 Perché la separazione è utile 102
44.4 Un esempio concreto 102

45 Linee guida di progettazione: mantenere i dati privati **103**
45.1 Perché i dati devono essere privati 103
45.2 Perché le operazioni devono essere pubbliche 103
45.3 Il problema dei dati pubblici 104
45.4 Il vantaggio di un'interfaccia stabile 104
45.5 Sintesi 104

46 Esercizio: riscrivere `point` usando coordinate polari **105**
46.1 Quali parti devono cambiare 105

47 Implementazione completa con coordinate polari **106**
47.1 Blocco 1: interfaccia pubblica e osservatori 106
47.2 Blocco 2: rappresentazione nascosta e modificatori 107

48 Cosa dimostra questo esercizio **108**

49 Classi e `struct` in C++: differenze e linee guida **108**
49.1 Differenza fondamentale: il livello di accesso predefinito 108
49.2 Quando usare `struct`: aggregati semplici 108
49.3 Quando usare `class`: incapsulamento e invarianti 109
49.4 Perché la scelta è importante 109

1 Perché sono necessari Classi e Struct

Questa sezione introduce la motivazione fondamentale che ha portato allo sviluppo della programmazione orientata agli oggetti (OOP). Per comprenderla appieno è utile partire dai limiti della programmazione procedurale tradizionale, che per molti anni è stata il paradigma dominante. L'obiettivo è mostrare come, all'aumentare della complessità dei programmi, diventi sempre più difficile mantenere separati dati e funzioni, e come ciò porti a errori, ridondanze e difficoltà di manutenzione.

1.1 Limitazioni della programmazione procedurale

La programmazione procedurale si basa su due elementi principali:

- **funzioni**, cioè blocchi di codice che eseguono operazioni specifiche;
- **dati separati**, cioè variabili e strutture dati che vengono manipolate dalle funzioni.

Questa separazione tra dati e funzioni è un tratto distintivo del paradigma procedurale. Tuttavia, quando i programmi crescono in dimensione e complessità, tale separazione inizia a mostrare limiti significativi.

1.1.1 Crescita della complessità

Quando un programma diventa più grande, aumenta anche il numero di variabili, strutture dati e funzioni che le manipolano. In un contesto procedurale:

- i dati possono essere accessibili da molte funzioni diverse;
- non esiste un meccanismo naturale per *legare* un insieme di dati alle funzioni che dovrebbero manipolarli;
- la responsabilità di mantenere la coerenza dei dati è distribuita in modo poco controllato.

Questo porta a una situazione in cui il programmatore deve ricordare manualmente quali funzioni operano su quali dati, e in quale ordine tali funzioni devono essere chiamate per evitare stati inconsistenti.

1.1.2 Maggiore probabilità di errori

Uno dei problemi più frequenti nella programmazione procedurale è la modifica errata dei dati. Poiché i dati sono spesso globali o comunque accessibili da molte parti del programma, è facile che:

- una funzione modifichi un valore in modo non previsto;
- un'altra funzione si aspetti che i dati siano in uno stato diverso;
- si generino bug difficili da tracciare, perché la causa può trovarsi in una funzione lontana da quella che manifesta l'errore.

La mancanza di un legame strutturato tra dati e comportamento rende difficile garantire l'integrità dello stato del programma.

1.2 La necessità di unire dati e comportamento

Per superare questi limiti, nasce l'esigenza di un paradigma che permetta di:

- **associare direttamente i dati alle funzioni** che li manipolano;
- **controllare l'accesso ai dati**, impedendo modifiche indesiderate;
- **organizzare il codice in unità logiche coerenti**, più facili da comprendere, mantenere ed estendere.

Questa esigenza porta alla nascita del concetto di **classe** e **struct**, che rappresentano entità capaci di contenere sia dati (attributi) sia funzioni (metodi). In questo modo, i dati non sono più entità isolate, ma diventano parte integrante di un'unità concettuale che ne definisce anche il comportamento.

Osservazione 1. *L'idea centrale è che i dati non dovrebbero "vivere da soli", ma dovrebbero essere accompagnati dalle operazioni che hanno senso per essi. Questo riduce drasticamente la possibilità di errori e rende il codice più leggibile e robusto.*

Concetto chiave

La programmazione orientata agli oggetti nasce come risposta diretta ai limiti della programmazione procedurale: serve a **legare insieme dati e comportamento**, creando strutture più sicure, modulari e facili da mantenere.

2 Che cos'è un Oggetto

Questa sezione introduce uno dei concetti fondamentali della programmazione orientata agli oggetti: l'**oggetto**. Comprendere cosa sia un oggetto, quali elementi lo compongono e perché rappresenti un'unità concettuale così potente è essenziale per costruire programmi robusti, modulari e facilmente estendibili.

2.1 Stato e comportamento in un'unica entità

Un oggetto è definito come un'**istanza** di una classe o di una struct. Il termine "istanza" indica che l'oggetto è una concretizzazione, un esemplare reale e utilizzabile di un modello astratto (la classe o la struct). Se la classe è il progetto, l'oggetto è l'edificio costruito a partire da quel progetto.

Ogni oggetto rappresenta un'entità concreta dotata di due componenti fondamentali:

- **uno stato**, costituito dai suoi dati interni (variabili membro, attributi);
- **un comportamento**, costituito dalle funzioni associate all'oggetto (metodi).

Questi due elementi non sono separati, ma convivono all'interno della stessa entità. L'oggetto diventa così un contenitore coerente che racchiude sia le informazioni sia le operazioni che hanno senso per quelle informazioni.

2.1.1 Lo stato dell'oggetto

Lo stato di un oggetto è l'insieme dei valori che i suoi attributi assumono in un determinato momento. Poiché ogni oggetto è un'istanza distinta, ciascuno possiede il proprio stato indipendente dagli altri oggetti della stessa classe.

Osservazione 2. *Due oggetti della stessa classe condividono la struttura dei dati e il comportamento, ma non i valori specifici dei dati. Questo permette di modellare entità simili ma non identiche.*

2.1.2 Il comportamento dell'oggetto

Il comportamento è definito dalle funzioni (metodi) che operano sui dati dell'oggetto. Queste funzioni sono parte integrante della classe e hanno accesso diretto allo stato interno dell'oggetto.

Il comportamento permette all'oggetto di:

- modificare il proprio stato;
- fornire informazioni sul proprio stato;
- interagire con altri oggetti;
- eseguire operazioni coerenti con il ruolo che l'oggetto rappresenta.

2.2 Unità concettuale: dati e operazioni insieme

Gli oggetti combinano dati e operazioni in un'unica unità logica. Questo rappresenta un cambiamento radicale rispetto alla programmazione procedurale, dove dati e funzioni sono entità separate.

Idea fondamentale

Un oggetto non è solo un contenitore di dati, né solo un insieme di funzioni: è l'unione coerente di entrambi. Questa unione riflette il modo naturale in cui percepiamo le entità nel mondo reale.

2.2.1 Parallelismo con il mondo reale

Il paradigma orientato agli oggetti nasce anche dall'osservazione che, nella vita quotidiana, pensiamo alle entità come a combinazioni di:

- **caratteristiche** (stato): colore, dimensioni, posizione, quantità;
- **azioni possibili** (comportamento): muoversi, cambiare colore, crescere, interagire.

Ad esempio, un "automobile" nel mondo reale ha:

- uno stato: velocità attuale, livello carburante, marcia inserita;
- un comportamento: accelerare, frenare, sterzare.

Un oggetto software modellato come automobile segue la stessa logica: possiede attributi e metodi che riflettono le caratteristiche e le azioni dell'entità reale.

Osservazione 3. *Questo parallelismo rende la progettazione più intuitiva e permette di costruire sistemi complessi partendo da componenti concettualmente semplici e ben definiti.*

3 Dal Dato all'Oggetto

Questa sezione approfondisce il passaggio concettuale fondamentale che porta dalla semplice gestione di dati isolati alla costruzione di oggetti completi, dotati sia di informazioni sia delle operazioni che permettono di manipolarle. Questo passaggio rappresenta il cuore della programmazione orientata agli oggetti e costituisce il punto di svolta rispetto al paradigma procedurale.

3.1 Raggruppare dati e comportamento in tipi

Il paradigma orientato agli oggetti introduce un'idea chiave: **dati correlati e funzioni che operano su tali dati devono essere raggruppati insieme all'interno di un unico tipo**. Questo tipo può essere una **classe** o una **struct**, a seconda del linguaggio e delle convenzioni adottate.

L'obiettivo è superare la frammentazione tipica della programmazione procedurale, dove:

- i dati sono definiti in un punto del programma,
- le funzioni che li manipolano sono definite altrove,
- non esiste un legame strutturale che garantisca coerenza e correttezza.

Classi e struct risolvono questo problema fornendo un contenitore logico che unisce:

- **dati correlati**, cioè attributi che descrivono lo stato dell'entità;
- **funzioni associate**, cioè metodi che definiscono il comportamento dell'entità.

Osservazione 4. *Il concetto di "tipo" in OOP non è più solo una categoria di valori (come* `int` *o* `double`*), ma diventa una rappresentazione astratta di un concetto, completo di struttura e comportamento.*

3.2 Vantaggi dell'approccio orientato agli oggetti

Raggruppare dati e funzioni in un'unica entità porta benefici significativi, soprattutto quando i programmi crescono in dimensione e complessità.

3.2.1 Miglioramento della leggibilità

Quando dati e operazioni sono collocati insieme:

- il codice diventa più intuitivo;
- è più facile capire quali funzioni appartengono a quali dati;
- la struttura del programma rispecchia la logica del problema.

Questo rende il codice più accessibile sia per chi lo scrive sia per chi lo legge successivamente.

3.2.2 Riduzione degli errori

L'associazione tra dati e comportamento riduce drasticamente la possibilità di modifiche indesiderate allo stato interno di un'entità. Le classi permettono infatti di:

- controllare l'accesso ai dati tramite meccanismi come `public`, `private` e `protected`;
- garantire che i dati vengano manipolati solo attraverso funzioni progettate per farlo correttamente;
- mantenere invarianti logici e condizioni di validità.

Principio di incapsulamento

L'incapsulamento è la pratica di racchiudere dati e funzioni all'interno di un'unica entità, controllando l'accesso allo stato interno. Questo principio è uno dei pilastri della programmazione orientata agli oggetti.

3.2.3 Modellazione naturale delle entità reali

L'approccio orientato agli oggetti permette di modellare concetti del mondo reale in modo diretto e intuitivo. Ogni entità può essere rappresentata come un tipo che possiede:

- caratteristiche (dati),
- capacità o azioni (funzioni).

Questo parallelismo rende più semplice progettare sistemi complessi, perché la struttura del codice riflette la struttura del dominio del problema.

3.3 Definire tipi che rappresentano concetti

La programmazione orientata agli oggetti inizia definendo **tipi** che rappresentano concetti rilevanti per il problema da risolvere. Ogni tipo diventa un modello astratto che descrive:

- quali dati caratterizzano il concetto;
- quali operazioni hanno senso per quel concetto;
- quali regole devono essere rispettate per mantenere la coerenza.

Osservazione 5. *Progettare un buon tipo significa identificare quali aspetti di un concetto devono essere rappresentati nel software e quali operazioni devono essere rese disponibili per manipolarlo. Questo processo è alla base della progettazione orientata agli oggetti.*

4 Scrivere le Classi

La costruzione di classi rappresenta il passaggio operativo attraverso cui la programmazione orientata agli oggetti prende forma concreta. Dopo aver compreso perché sia necessario unire dati e comportamento e come gli oggetti modellino entità reali, il passo successivo consiste nel definire tipi che incorporano struttura interna e funzionalità esterna. Questa sezione chiarisce come si passi da un insieme di semplici funzioni a tipi completi che rappresentano concetti, entità e relazioni.

4.1 Dalle funzioni ai tipi orientati agli oggetti

L'obiettivo principale della programmazione orientata agli oggetti è superare l'approccio basato su "sole funzioni" tipico del paradigma procedurale. In un programma tradizionale, le funzioni operano su dati che spesso non appartengono a un contesto specifico, generando dispersione e mancanza di coesione.

La definizione di una classe permette di:

- associare dati e operazioni in un'unica struttura coerente;
- modellare concetti complessi come entità dotate di stato e comportamento;
- creare tipi personalizzati che si comportano come veri e propri mattoni concettuali del programma.

Osservazione 6. *Il passaggio da funzioni isolate a tipi strutturati è ciò che consente di costruire sistemi software scalabili, modulari e più vicini al modo in cui ragioniamo sulle entità del mondo reale.*

4.2 Che cosa definisce una classe

Una classe stabilisce due aspetti fondamentali:

- **un'interfaccia**, cioè ciò che gli utenti della classe possono fare con gli oggetti che essa genera;
- **un'implementazione**, cioè il modo in cui la classe realizza internamente il proprio comportamento.

Questa distinzione è cruciale. L'interfaccia rappresenta la parte visibile e utilizzabile dall'esterno, mentre l'implementazione è la parte nascosta che contiene i dettagli operativi. Separare questi due livelli permette di:

- proteggere i dati interni da modifiche indesiderate;
- garantire che gli oggetti vengano utilizzati correttamente;
- modificare l'implementazione senza alterare il modo in cui la classe viene usata.

Interfaccia vs Implementazione

L'interfaccia descrive *cosa* un oggetto può fare; l'implementazione descrive *come* lo fa. Questa separazione è uno dei principi cardine della progettazione orientata agli oggetti.

4.3 Rendere privata la rappresentazione ed esporre il comportamento

Il principio chiave nella definizione delle classi è mantenere la rappresentazione interna dei dati **privata** ed esporre invece il comportamento attraverso metodi **pubblici**. Questo approccio è noto come **incapsulamento**.

4.3.1 Perché la rappresentazione deve essere privata

Rendere privati i dati interni permette di:

- impedire modifiche dirette e potenzialmente errate allo stato dell'oggetto;
- garantire che ogni cambiamento avvenga attraverso metodi controllati;
- mantenere invarianti logici e condizioni di validità;
- evitare che parti esterne del programma dipendano da dettagli implementativi.

4.3.2 Perché il comportamento deve essere pubblico

I metodi pubblici costituiscono l'interfaccia dell'oggetto e permettono agli utenti di:

- interrogare lo stato dell'oggetto;
- modificarlo in modo sicuro;
- attivare operazioni coerenti con il ruolo dell'oggetto.

Osservazione 7. *L'utente della classe non deve conoscere la struttura interna dei dati, ma solo quali operazioni può eseguire. Questo rende il codice più robusto e più facile da mantenere.*

5 Classi e Struct

La distinzione tra `class` e `struct` in C++ è spesso fonte di domande per chi inizia a studiare la programmazione orientata agli oggetti. Sebbene i due costrutti condividano quasi completamente la stessa struttura sintattica e semantica, esistono differenze di default che influenzano il modo in cui vengono utilizzati nella pratica. Questa sezione chiarisce tali differenze e spiega le linee guida stilistiche comunemente adottate.

5.1 La differenza pratica tra class e struct

Nel linguaggio C++, `class` e `struct` sono quasi identici. Entrambi permettono di definire:

- dati (attributi),
- funzioni (metodi),
- costruttori e distruttori,
- ereditarietà,
- incapsulamento tramite specificatori di accesso.

La differenza principale riguarda il **valore di default dello specificatore di accesso**:

- in una `struct`, i membri sono **pubblici** per default;
- in una `class`, i membri sono **privati** per default.

Questa differenza, pur essendo minima dal punto di vista tecnico, ha un impatto significativo sul modo in cui i due costrutti vengono utilizzati nella progettazione del software.

Default di accesso

- `struct`: `public` di default.
- `class`: `private` di default.

5.2 Linee guida stilistiche

Nella pratica della programmazione moderna, esiste una convenzione ampiamente accettata che distingue l'uso di `struct` e `class` in base alla natura del tipo che si vuole modellare.

5.2.1 Quando usare struct

Si utilizza `struct` quando si vuole rappresentare un semplice contenitore di dati, senza invarianti complessi o logiche di comportamento articolate. In altre parole, una `struct` è adatta a:

- dati "semplici" o "plain old data" (POD),
- tipi che non richiedono incapsulamento rigoroso,
- strutture usate principalmente per trasportare informazioni.

5.2.2 Quando usare class

Si utilizza `class` quando il tipo deve:

- mantenere invarianti logici,
- controllare l'accesso ai dati,
- esporre un comportamento ben definito tramite metodi pubblici,
- nascondere dettagli implementativi.

Osservazione 8. *La scelta tra `class` e `struct` non è imposta dal linguaggio, ma riflette l'intenzione del progettista. Usare `class` comunica che il tipo possiede logica interna e regole da rispettare; usare `struct` comunica che il tipo è un contenitore di dati senza complessità.*

6 Anatomia di una Classe

L'analisi della struttura interna di una classe permette di comprendere come dati e comportamento vengano organizzati in un'unità coerente. Un esempio minimale è sufficiente per illustrare i concetti fondamentali: costruttori, metodi, incapsulamento e distinzione tra interfaccia pubblica e rappresentazione privata.

6.1 Un esempio minimale

Il frammento seguente mostra una classe `counter`, progettata per rappresentare un semplice contatore. L'obiettivo è evidenziare come una classe definisca sia lo stato interno sia le operazioni che agiscono su tale stato.

Codice C++: struttura di una classe

```cpp
// Dichiarazione della classe "counter"
class counter {                        // Definizione del tipo "counter"

// Sezione pubblica: interfaccia accessibile agli utenti della classe
public:

    counter() : value_{0} {}           // Costruttore: inizializza il contatore a 0

    void increment() {                 // Metodo pubblico: incrementa il valore
        ++value_;                      // Operazione di incremento sul dato interno
    }

    int value() const {                // Metodo osservatore: restituisce il valore
        return value_;                 // Restituisce lo stato interno senza modificarlo
    }

// Sezione privata: rappresentazione interna nascosta
private:
    int value_;                        // Dato interno: lo stato del contatore
};
```

6.2 Output atteso

Esempio di utilizzo e output

```
// Esempio di uso:
counter c;        // c.value_ = 0
c.increment();    // c.value_ = 1
c.increment();    // c.value_ = 2

std::cout << c.value();
// Output: 2
```

6.3 Spiegazione dettagliata del codice

L'esempio contiene tutti gli elementi essenziali di una classe C++.

6.3.1 La dichiarazione della classe

La riga iniziale:

```
class counter {
```

introduce un nuovo tipo definito dall'utente. Questo tipo rappresenta un concetto: un contatore che mantiene un valore interno e permette di incrementarlo.

6.3.2 La sezione pubblica: l'interfaccia

La parola chiave `public:` introduce la parte dell'interfaccia visibile all'esterno. Tutti i membri dichiarati in questa sezione possono essere utilizzati da chiunque possieda un oggetto di tipo `counter`.

Gli elementi pubblici sono:

- **il costruttore**

  ```
  counter() : value_{0} {}
  ```

  Inizializza il membro privato `value_` a 0. L'uso dell'inizializzazione tramite lista (`: value_0`) è la forma idiomatica in C++ per inizializzare i membri.

- **il metodo `increment()`** Incrementa il valore interno del contatore. L'operazione `++value_` modifica lo stato dell'oggetto.

- **il metodo `value() const`** Restituisce il valore corrente del contatore. La presenza della parola chiave `const` indica che il metodo non modifica lo stato interno dell'oggetto. Questo tipo di metodo è chiamato **osservatore**.

6.3.3 La sezione privata: la rappresentazione interna

La parola chiave `private:` introduce la parte nascosta della classe. Qui si trova:

`int value_;` È il dato interno che rappresenta lo stato del contatore. Essendo privato, non può essere modificato direttamente dall'esterno.

Principio di incapsulamento

La rappresentazione interna (`value_`) è nascosta e protetta. L'utente può solo usare i metodi pubblici (`increment()`, `value()`). Questo garantisce che lo stato dell'oggetto rimanga sempre valido.

6.4 Osservatori e metodi `const`

Il metodo:

```
int value() const
```

è un esempio di **osservatore**, cioè una funzione che:

- legge lo stato dell'oggetto,
- non lo modifica,
- può essere chiamata anche su oggetti costanti.

La parola chiave `const` dopo la firma del metodo è un vincolo semantico: garantisce che il metodo non alteri alcun membro dell'oggetto.

Osservazione 9. *L'uso corretto di metodi `const` è fondamentale per progettare classi sicure e prevedibili, perché permette al compilatore di verificare automaticamente che lo stato non venga modificato in modo improprio.*

7 Ogni Oggetto Possiede i Propri Membri Dati

Uno dei principi fondamentali della programmazione orientata agli oggetti è che **ogni oggetto è un'istanza indipendente** della classe da cui è stato creato. Ciò significa che ogni oggetto possiede una propria copia dei membri dati, e che i metodi operano sempre sull'istanza specifica su cui vengono invocati. Questa caratteristica permette di modellare entità multiple che condividono struttura e comportamento, ma mantengono stati distinti.

7.1 Oggetti come istanze indipendenti

Quando si crea un oggetto a partire da una classe, il compilatore alloca memoria per i membri dati di quell'oggetto. Se si creano due oggetti distinti, ciascuno avrà la propria area di memoria per i dati interni. Questo implica che:

- ogni oggetto mantiene il proprio stato; - le modifiche fatte su un oggetto non influenzano gli altri; - i metodi operano sempre sullo stato dell'oggetto su cui vengono chiamati.

Concetto chiave

Gli oggetti condividono il comportamento (i metodi), ma non lo stato (i dati). Ogni oggetto ha la propria copia dei membri dati.

7.2 Esempio: due contatori indipendenti

Il seguente frammento di codice mostra due oggetti della classe `counter`. Ognuno possiede il proprio membro dati `value_`, e le operazioni effettuate su uno non influenzano l'altro.

Codice C++: oggetti indipendenti

```cpp
counter a{};        // Crea l'oggetto 'a' con value_ inizializzato a 0
counter b{};        // Crea l'oggetto 'b' con value_ inizializzato a 0

a.increment();      // Incrementa solo il valore interno di 'a'
b.increment();      // Incrementa solo il valore interno di 'b'
```

7.3 Output atteso

Esempio di output

```
std::cout << a.value();    // Output: 1
std::cout << b.value();    // Output: 1
```

7.4 Spiegazione dettagliata

Il comportamento osservato deriva direttamente dalla natura delle istanze:

- `counter a` crea un oggetto con il proprio `value_ = 0`. - `counter b` crea un secondo oggetto, anch'esso con `value_ = 0`, ma in un'area di memoria distinta. - `a.increment()` modifica solo lo stato dell'oggetto `a`. - `b.increment()` modifica solo lo stato dell'oggetto `b`.

I metodi non operano su variabili globali o condivise, ma sul membro dati dell'istanza su cui vengono invocati. Questo è possibile grazie al meccanismo implicito del puntatore `this`, che permette a ogni metodo di riferirsi ai dati dell'oggetto corrente.

Osservazione 10. *La separazione dello stato tra oggetti diversi è ciò che consente di modellare entità multiple e indipendenti nel programma, come più contatori, più finestre grafiche, più vettori, più giocatori in un videogioco, ecc.*

8 Il Nome della Classe è Conosciuto all'Interno della Definizione

All'interno della definizione di una classe, il nome della classe stessa è immediatamente visibile e utilizzabile a partire dalla parentesi graffa di apertura. Questa caratteristica è fondamentale perché permette alla classe di riferirsi a sé stessa come tipo, consentendo la definizione di costruttori di copia, metodi che restituiscono oggetti dello stesso tipo, funzioni statiche che creano istanze e molto altro.

8.1 Usare il tipo all'interno della propria dichiarazione

Il nome della classe diventa parte dello scope non appena la definizione inizia. Questo significa che, all'interno del corpo della classe, è possibile utilizzare il nome della classe come se fosse già completamente definito, anche se la definizione non è ancora terminata. Questo comportamento è essenziale per dichiarare:

- costruttori di copia,
- operatori di assegnazione,
- metodi che accettano o restituiscono oggetti dello stesso tipo,
- alias interni,
- tipi annidati.

Codice C++: uso del nome della classe all'interno della definizione

```
class counter {                                  // Inizio della definizione della classe

public:

    counter(counter const& other);               // Costruttore di copia: usa 'counter' come tipo

};                                               // Fine della definizione
```

8.2 Spiegazione dettagliata del codice

La riga:

```
counter(counter const& other);
```

dichiara un **costruttore di copia**, cioè una funzione speciale che permette di creare un nuovo oggetto `counter` copiando lo stato di un altro oggetto dello stesso tipo. Il punto cruciale è che il tipo `counter` è già riconosciuto dal compilatore, anche se la classe non è ancora completamente definita.

Questo è possibile perché:

- il nome della classe entra nello scope immediatamente dopo la parola chiave `class counter`,
- il compilatore può già trattare `counter` come un tipo incompleto,
- per dichiarare un costruttore di copia è sufficiente conoscere il nome del tipo, non la sua struttura interna.

8.3 Perché questo è importante

La possibilità di usare il nome della classe all'interno della sua stessa definizione permette di:

- definire costruttori di copia e operatori di assegnazione, - dichiarare metodi che restituiscono oggetti della stessa classe, - implementare pattern come il "metodo factory", - creare strutture dati ricorsive (ad esempio liste concatenate).

Concetto chiave

Il nome della classe è disponibile fin dall'inizio della definizione. Questo consente alla classe di riferirsi a sé stessa come tipo, anche prima che la definizione sia completa.

Osservazione 11. *Il costruttore di copia è solo uno dei casi più comuni in cui questa caratteristica è utilizzata. Ogni volta che un metodo deve manipolare o restituire oggetti dello stesso tipo, il nome della classe deve essere già visibile.*

9 Incapsulamento

L'incapsulamento è uno dei principi centrali della programmazione orientata agli oggetti. Consiste nel separare in modo netto ciò che una classe *offre* all'esterno (la sua interfaccia pubblica) da ciò che *realizza internamente* (la sua implementazione privata). Questo principio permette di proteggere lo stato dell'oggetto, garantire la coerenza interna e rendere il codice più leggibile e manutenibile.

9.1 Separare interfaccia e implementazione

L'idea fondamentale è distinguere tra:

- **ciò che la classe fa**: l'insieme delle operazioni disponibili agli utenti, dichiarate nella sezione `public`;
- **come la classe lo fa**: la rappresentazione interna dei dati e i dettagli operativi, collocati nella sezione `private`.

Questa separazione crea una barriera concettuale che impedisce agli utenti della classe di manipolare direttamente lo stato interno, riducendo il rischio di errori e preservando gli invarianti logici.

9.2 Protezione degli invarianti

Un invariante è una condizione che deve essere sempre vera per garantire la validità dell'oggetto. Se i dati interni fossero accessibili liberamente, sarebbe facile violare tali condizioni, portando l'oggetto in uno stato "impossibile" o incoerente.

L'incapsulamento:

- impedisce modifiche dirette allo stato interno;
- obbliga l'utente a passare attraverso metodi controllati;
- permette alla classe di verificare e mantenere le proprie regole interne.

Esempio concettuale

Un contatore non dovrebbe mai assumere valori negativi. Rendendo privato il membro dati e fornendo solo metodi controllati, la classe può impedire che ciò accada.

9.3 Accesso controllato ai dati

L'incapsulamento permette di definire con precisione quali operazioni sono consentite e quali no. I metodi pubblici fungono da "guardiani" dello stato interno:

- controllano i valori in ingresso,
- applicano logiche di validazione,
- mantengono la coerenza dell'oggetto.

Questo approccio riduce drasticamente la possibilità di errori accidentali o modifiche improprie.

9.4 Flessibilità nell'implementazione

Un vantaggio cruciale dell'incapsulamento è la possibilità di modificare l'implementazione interna senza cambiare il codice che utilizza la classe. Finché l'interfaccia pubblica rimane invariata, gli utenti non devono modificare nulla.

Questo consente di:

- ottimizzare le prestazioni,
- cambiare la struttura dei dati interni,
- aggiungere controlli o logiche supplementari,
- correggere bug senza impatti esterni.

Osservazione 12. *L'incapsulamento è ciò che permette di progettare classi robuste e stabili nel tempo, anche quando l'implementazione evolve.*

9.5 Miglioramento della leggibilità

Separare ciò che è pubblico da ciò che è privato rende la classe più chiara da leggere e comprendere. L'utente vede immediatamente:

- **cosa la classe permette di fare** (sezione `public`);
- **come la classe realizza tali operazioni** (sezione `private`).

Questa distinzione migliora la comprensione del codice e facilita la manutenzione.

10 Funzioni Membro

Le funzioni membro costituiscono il meccanismo attraverso cui un oggetto espone il proprio comportamento. A differenza delle funzioni "libere" tipiche della programmazione procedurale, una funzione membro è sempre associata a un'istanza specifica della classe e opera sul suo stato interno. Questo legame tra funzione e oggetto è uno degli elementi che rendono la programmazione orientata agli oggetti più espressiva e naturale.

10.1 Funzioni che operano sugli oggetti

Una funzione membro è una funzione definita all'interno della classe. La sua caratteristica fondamentale è che:

- ha accesso diretto ai membri dati dell'oggetto;
- viene invocata su un'istanza specifica;
- modifica o osserva lo stato di quell'istanza.

Questo significa che il comportamento non è più separato dai dati, ma è parte integrante dell'oggetto stesso.

Concetto chiave

Una funzione membro rappresenta un'operazione che un oggetto può compiere. L'oggetto è il soggetto dell'azione, non un semplice parametro.

10.2 Invocazione delle funzioni membro

Le funzioni membro non vengono chiamate in modo isolato, ma sempre attraverso un oggetto. La sintassi:

```
oggetto.funzione();
```

indica che la funzione viene eseguita sullo stato interno di quell'oggetto.

Codice C++: chiamata di funzioni membro

```
counter c;          // Crea un oggetto 'c' di tipo counter
c.increment();      // Chiama increment() sull'oggetto 'c'
```

Ogni chiamata agisce sull'istanza specifica su cui viene invocata. Se esistono più oggetti della stessa classe, ciascuno mantiene il proprio stato e le funzioni membro operano indipendentemente.

10.3 Esempio con più oggetti

Codice C++: funzioni membro su istanze diverse

```
counter a{};        // Oggetto 'a' con value_ = 0
counter b{};        // Oggetto 'b' con value_ = 0

a.increment();      // Modifica solo 'a'
b.increment();      // Modifica solo 'b'
```

10.4 Spiegazione dettagliata

Ogni oggetto possiede la propria copia dei membri dati. Quando si invoca una funzione membro:

- il compilatore passa implicitamente un puntatore all'oggetto corrente, chiamato `this`;
- la funzione opera sui dati puntati da `this`;
- lo stato modificato riguarda solo l'istanza su cui la funzione è stata chiamata.

Questo meccanismo permette di modellare entità multiple e indipendenti, ognuna con il proprio comportamento coerente.

Osservazione 13. *Il fatto che ogni funzione membro operi su un'istanza specifica è ciò che rende gli oggetti autonomi e capaci di mantenere uno stato distinto. Questo è essenziale per modellare sistemi complessi composti da molte entità interagenti.*

11 Il Puntatore `this`

Il puntatore `this` è un meccanismo fondamentale della programmazione orientata agli oggetti in C++. Ogni funzione membro riceve automaticamente un puntatore all'oggetto sul quale è stata invocata. Questo permette ai metodi di accedere e modificare lo stato interno dell'istanza specifica, garantendo che ogni oggetto mantenga un comportamento coerente con il proprio stato.

11.1 Come le funzioni membro accedono all'oggetto

Ogni volta che una funzione membro viene chiamata, il compilatore passa implicitamente un puntatore chiamato `this`. Questo puntatore:

- contiene l'indirizzo dell'oggetto su cui la funzione è stata invocata;
- permette di accedere ai membri dati e ai metodi dell'istanza corrente;
- rende possibile distinguere tra oggetti diversi, anche se condividono lo stesso codice.

Concetto chiave

`this` punta sempre all'oggetto "corrente", cioè quello che sta eseguendo la funzione membro.

11.2 Accesso ai membri tramite `this`

All'interno di una funzione membro, quando si scrive:

```
value_
```

il compilatore interpreta automaticamente questa espressione come:

```
this->value_
```

Questo significa che ogni accesso ai membri dati è, in realtà, un accesso ai dati dell'oggetto puntato da `this`. Il programmatore può omettere `this->` per comodità, ma il significato rimane invariato.

11.3 Esempio: uso implicito e esplicito di `this`

Codice C++: uso del puntatore `this`

```
void counter::increment() {      // Definizione del metodo
    increment()
    ++value_;                    // Accesso implicito: equivale
        a ++this->value_;
}
```

11.4 Spiegazione dettagliata del codice

La funzione membro `increment()` opera sul membro dati `value_`. Quando si scrive:

```
++value_;
```

il compilatore lo interpreta come:

```
++(this->value_);
```

Questo comportamento è essenziale per garantire che:

- ogni oggetto modifichi solo il proprio stato;
- le funzioni membro siano riutilizzabili per tutte le istanze della classe;
- il codice rimanga leggibile senza dover scrivere esplicitamente `this->` ogni volta.

11.5 Perché `this` è importante

Il puntatore `this` permette di:

- distinguere tra membri dati e parametri con lo stesso nome;
- restituire l'oggetto corrente (pattern fluent interface);
- implementare operatori come l'assegnazione;
- garantire che ogni metodo lavori sull'istanza corretta.

Osservazione 14. *Il puntatore `this` è implicito, ma comprenderne il funzionamento è essenziale per capire come le funzioni membro interagiscono con lo stato dell'oggetto e perché ogni istanza mantiene un comportamento indipendente.*

12 Accesso ai Membri e Precedenza degli Operatori

La comprensione della precedenza degli operatori è essenziale per interpretare correttamente le espressioni che coinvolgono oggetti, puntatori e funzioni membro. In particolare, quando si lavora con il puntatore `this`, è importante sapere come il compilatore valuta le espressioni che accedono ai membri dell'oggetto.

12.1 Interpretazione delle espressioni

L'operatore punto `.` e l'operatore freccia `->` sono utilizzati per accedere ai membri di un oggetto. Tuttavia, quando si lavora con puntatori, entra in gioco anche l'operatore di dereferenziazione `*`. La precedenza tra questi operatori determina come il compilatore interpreta un'espressione.

L'operatore `.` ha una precedenza più alta rispetto all'operatore `*`. Questo significa che, in un'espressione come:

```
(*this).value_
```

il compilatore valuta prima `*this`, ottenendo l'oggetto, e poi applica l'operatore punto per accedere al membro `value_`.

12.2 Uso di `this->` per chiarezza

Sebbene sia possibile scrivere:

```
(*this).value_ = value;
```

la forma più leggibile e idiomatica in C++ è:

```
this->value_ = value;
```

Questa sintassi rende esplicito che si sta accedendo a un membro dell'oggetto puntato da `this`, migliorando la chiarezza del codice.

Codice C++: precedenza degli operatori

```
this->value_ = value;       // Accesso chiaro e diretto al membro

(*this).value_ = value;     // Equivalente, ma meno leggibile
```

12.3 Spiegazione dettagliata

L'operatore `->` è, in realtà, una scorciatoia per la combinazione:

```
(*puntatore).membro
```

Quando si scrive `this->value_`, il compilatore esegue due operazioni:

- dereferenzia `this`, ottenendo l'oggetto corrente;
- accede al membro `value_` tramite l'operatore punto.

La precedenza degli operatori garantisce che:

- `.` e `->` vengano valutati prima di `*`;
- l'espressione `(*this).value_` sia interpretata correttamente.

Osservazione 15. *Anche se entrambe le forme sono corrette, l'uso di `this->` è preferito perché comunica immediatamente l'intenzione del programmatore e riduce la possibilità di errori di lettura o interpretazione.*

13 Funzioni Membro `const`

Le funzioni membro dichiarate come `const` svolgono un ruolo essenziale nella progettazione di classi sicure e prevedibili. Esse rappresentano un impegno formale: garantiscono che l'oggetto sul quale vengono chiamate non verrà modificato. Questo vincolo permette al compilatore di applicare controlli più rigorosi e consente al programmatore di esprimere chiaramente l'intenzione di non alterare lo stato interno.

13.1 Garanzie sullo stato dell'oggetto

Una funzione membro marcata come `const` promette che non modificherà alcun membro dati dell'oggetto. Questa promessa è verificata dal compilatore, che impedirà qualsiasi tentativo di alterare lo stato interno all'interno della funzione.

Dichiarare una funzione come `const` significa che essa è un'operazione di sola lettura, un osservatore che non cambia lo stato dell'istanza.

13.2 Il puntatore `this` in funzioni `const`

Quando una funzione membro è dichiarata `const`, il puntatore implicito `this` viene trattato come:

```
const counter*
```

Questo significa che:

- `this` punta all'oggetto corrente,
- ma l'oggetto è considerato costante,
- quindi non è possibile modificarne i membri dati,
- e non è possibile chiamare metodi non-`const`.

Effetto sul puntatore `this`

In una funzione membro `const`, `this` è un puntatore a oggetto costante. Qualsiasi tentativo di modificare lo stato interno genera un errore di compilazione.

13.3 Conseguenze pratiche

Le funzioni membro `const` hanno due implicazioni fondamentali:

- **non possono modificare i membri dati**, né direttamente né tramite metodi non-`const`;
- **possono chiamare solo altre funzioni membro `const`**, perché solo queste garantiscono di non alterare lo stato.

Questo meccanismo permette di distinguere chiaramente tra:

- metodi osservatori (che non modificano lo stato),
- metodi mutatori (che modificano lo stato).

13.4 Esempio illustrativo

Codice C++: funzione membro `const`

```cpp
int counter::value() const {      // Metodo osservatore
    return value_;                // Accesso consentito: non
       modifica lo stato
}
```

In questo esempio, il metodo `value()` è dichiarato `const`, quindi:

- può leggere `value_`,
- non può modificarlo,
- può essere chiamato anche su oggetti dichiarati come `const`.

13.5 Importanza progettuale

L'uso corretto delle funzioni `const` migliora la qualità del codice perché:

- rende esplicite le intenzioni del programmatore,
- permette al compilatore di prevenire errori,
- facilita la comprensione della classe distinguendo chiaramente tra lettura e modifica dello stato,
- consente di utilizzare oggetti in contesti che richiedono immutabilità (ad esempio, come parametri costanti).

Osservazione 16. *Una classe ben progettata presenta una chiara separazione tra metodi osservatori e metodi mutatori. L'uso sistematico di funzioni* `const` *è un segno distintivo di codice C++ robusto e professionale.*

14 Osservatori e Modificatori

La distinzione tra osservatori e modificatori è fondamentale per progettare classi chiare, sicure e coerenti. Ogni metodo dovrebbe esprimere chiaramente se il suo scopo è leggere lo stato dell'oggetto oppure modificarlo. Questa separazione migliora la leggibilità, facilita la manutenzione e permette al compilatore di applicare controlli più rigorosi.

14.1 Ispezionare lo stato: gli osservatori

Gli osservatori sono funzioni membro che leggono lo stato dell'oggetto senza modificarlo. Le loro caratteristiche principali sono:

- ispezionano lo stato interno;
- non alterano alcun membro dati;
- sono quasi sempre dichiarati `const`;
- possono essere chiamati anche su oggetti costanti.

Esempio di osservatore

```
int value() const;      // Osservatore: non modifica lo stato
```

Dichiarare un metodo come `const` garantisce che il compilatore impedirà qualsiasi tentativo di modifica dello stato interno, rendendo il comportamento del metodo più sicuro e prevedibile.

14.2 Modificare lo stato: i modificatori

I modificatori sono funzioni membro che cambiano lo stato dell'oggetto. Le loro caratteristiche principali sono:

- alterano uno o più membri dati;
- non possono essere dichiarati `const`;
- rappresentano operazioni che trasformano l'oggetto.

Esempio di modificatore

```
void increment();       // Modificatore: cambia lo stato
    interno
```

Questi metodi sono essenziali per modellare comportamenti dinamici, come aggiornamenti, trasformazioni o cambiamenti di stato.

14.3 Perché la distinzione è importante

Separare chiaramente osservatori e modificatori porta diversi vantaggi:

- rende immediatamente evidente quali metodi sono sicuri da chiamare senza effetti collaterali;
- permette al compilatore di applicare controlli più rigorosi grazie alla parola chiave `const`;
- migliora la leggibilità e la documentazione implicita della classe;
- facilita l'uso dell'oggetto in contesti che richiedono immutabilità.

Principio progettuale

Un buon design orientato agli oggetti distingue nettamente tra metodi che leggono lo stato e metodi che lo modificano. Questa separazione rende la classe più chiara, più sicura e più facile da mantenere.

15 Nomenclatura dei Membri

La scelta dei nomi per i membri dati è una parte importante della progettazione di una classe. Una convenzione molto diffusa nel C++ moderno consiste nel far terminare i nomi dei membri dati con un carattere di underscore finale _. Questa scelta, pur semplice, porta benefici significativi in termini di leggibilità, coerenza e prevenzione degli errori.

15.1 Convenzione comune: membri dati con suffisso _

Molti progetti C++ adottano la convenzione di nominare i membri dati in questo modo:

- `int value_;`
- `std::string name_;`

Il suffisso _ rende immediatamente riconoscibile che si tratta di un membro dati dell'oggetto e non di una variabile locale o di un parametro.

15.2 Perché questa convenzione è utile

L'uso del suffisso _ offre diversi vantaggi pratici:

- **Evita conflitti di nomi** con parametri o variabili locali. Questo è particolarmente utile nei costruttori, dove i parametri spesso hanno lo stesso nome dei membri.
- **Rende immediatamente riconoscibile l'accesso ai membri**. Quando si legge il codice, è chiaro quali variabili appartengono all'oggetto.
- **Funziona perfettamente con le liste di inizializzazione dei costruttori**. La distinzione tra parametro e membro è evidente e non richiede l'uso esplicito di `this->`.

15.3 Esempio con lista di inizializzazione

L'esempio seguente mostra un costruttore che utilizza la convenzione del suffisso _:

Codice C++: inizializzazione chiara dei membri

```
counter(int value) : value_{value} {}
```

In questo caso:

- `value` è il parametro del costruttore,
- `value_` è il membro dati dell'oggetto.

La distinzione è immediata e non richiede l'uso di `this->value_ = value;`.

15.4 Impatto sulla leggibilità e sulla manutenzione

Questa convenzione migliora la qualità del codice perché:

- riduce l'ambiguità nei costruttori e nei metodi;
- rende più semplice navigare il codice e capire quali variabili appartengono all'oggetto;
- evita errori sottili dovuti a shadowing dei nomi;
- mantiene uno stile coerente in tutto il progetto.

Osservazione 17. *Le convenzioni di naming non sono obbligatorie, ma adottarne una chiara e coerente è un segno distintivo di codice professionale e facilita la collaborazione in team.*

16 Dichiarazioni Multiple di Membri Dati

Il linguaggio C++ permette di dichiarare più membri dati all'interno di un'unica istruzione. Sebbene questa pratica sia perfettamente legale dal punto di vista sintattico, è spesso scoraggiata nello stile moderno perché può ridurre la leggibilità e rendere meno chiara la struttura interna della classe.

16.1 Una pratica consentita ma sconsigliata

È possibile dichiarare più membri dati in un'unica riga, come nell'esempio seguente:

Codice C++: dichiarazione multipla

```
class point {
    double x_, y_;
};
```

Questa forma è compatta e funziona correttamente, ma presenta alcuni svantaggi:

- riduce la chiarezza visiva della classe;
- rende meno immediato individuare e documentare ciascun membro;
- può creare ambiguità quando i tipi diventano più complessi;
- non si presta bene a commenti puntuali su ogni membro.

16.2 Perché lo stile moderno la scoraggia

La tendenza attuale nella scrittura di codice C++ leggibile e manutenibile è quella di dichiarare ogni membro su una riga separata. Questo approccio:

- migliora la leggibilità, soprattutto in classi più grandi;
- facilita l'aggiunta di commenti descrittivi per ogni membro;
- rende più chiara la struttura interna della classe;
- evita errori quando si modificano i tipi o si aggiungono qualificatori.

Stile consigliato

```
class point {
    double x_;
    double y_;
};
```

La versione espansa è più chiara e si integra meglio con convenzioni di naming, strumenti di analisi statica e formattatori automatici.

16.3 Impatto sulla progettazione

Anche se la dichiarazione multipla non è un errore, adottare uno stile più esplicito contribuisce a:

- mantenere il codice più leggibile nel lungo periodo;
- ridurre la possibilità di errori durante refactoring o estensioni;
- rendere la classe più auto-documentante.

Questa attenzione alla forma è particolarmente importante in progetti collaborativi o in codice destinato a durare nel tempo.

17 Costruttori

I costruttori sono il meccanismo attraverso cui un oggetto viene inizializzato nel momento stesso in cui nasce. Ogni classe dovrebbe definire almeno un costruttore che garantisca che l'oggetto inizi la sua vita in uno stato valido, rispettando gli invarianti della classe. Senza un costruttore adeguato, i membri integrali potrebbero rimanere non inizializzati e l'oggetto potrebbe trovarsi in condizioni incoerenti o addirittura pericolose.

17.1 Creare oggetti validi fin dall'inizio

Un costruttore è una funzione speciale che ha il compito di:

- inizializzare i membri dati dell'oggetto;
- stabilire gli invarianti della classe;
- garantire che l'oggetto sia immediatamente utilizzabile.

Il costruttore viene chiamato automaticamente quando un oggetto viene creato, senza necessità di invocarlo esplicitamente.

Idea fondamentale

Il costruttore assicura che ogni oggetto inizi la sua esistenza in uno stato valido e coerente.

17.2 Cosa accade senza un costruttore

Se una classe non definisce un costruttore, il compilatore ne genera uno di default. Tuttavia, questo costruttore:

- non inizializza i membri integrali (come `int`, `double`);
- non impone alcun invariante;
- lascia potenzialmente l'oggetto in uno stato indefinito.

Questo può portare a comportamenti imprevedibili, bug difficili da individuare e violazioni degli invarianti logici della classe.

17.3 Perché i costruttori sono essenziali

I costruttori rendono semplice e sicuro creare oggetti correttamente inizializzati. Definire un costruttore esplicito permette di:

- evitare valori non inizializzati;
- imporre condizioni di validità;
- rendere chiaro come deve essere costruito un oggetto;
- migliorare la robustezza del codice.

17.4 Esempio concettuale

Una classe come `counter` può garantire che il suo valore interno inizi sempre da zero:

Codice C++: costruttore che stabilisce un invariante

```cpp
class counter {
public:
    counter() : value_{0} {}   // L'oggetto nasce sempre in stato valido
private:
    int value_;
};
```

In questo esempio:

- `value_` è sempre inizializzato; - l'oggetto non può trovarsi in uno stato indefinito; - l'invariante "il contatore ha un valore valido" è rispettato.

17.5 Impatto sulla progettazione

L'uso sistematico dei costruttori contribuisce a:

- ridurre gli errori legati a variabili non inizializzate;
- rendere più chiaro il contratto d'uso della classe;
- facilitare l'estensione futura della classe;
- mantenere invarianti robusti e verificabili.

Osservazione 18. *Un oggetto che nasce in uno stato valido è più facile da usare, più sicuro e più prevedibile. La progettazione orientata agli oggetti si basa proprio su questa garanzia.*

18 Liste di Inizializzazione dei Membri

La lista di inizializzazione del costruttore è uno degli strumenti più importanti del C++ moderno per garantire che gli oggetti vengano costruiti in modo corretto, efficiente e coerente con gli invarianti della classe. A differenza delle assegnazioni nel corpo del costruttore, la lista di inizializzazione permette di inizializzare i membri *prima* dell'esecuzione del corpo stesso, assicurando che ogni membro nasca già nello stato desiderato.

18.1 Inizializzazione prima del corpo del costruttore

Una lista di inizializzazione ha la forma:

```
counter() :  value_{0} {}
```

e viene eseguita prima dell'ingresso nel corpo del costruttore. Questo significa che:

- i membri vengono costruiti direttamente con i valori desiderati;
- non si verifica alcuna inizializzazione predefinita seguita da assegnazione;
- l'oggetto inizia la sua vita già in uno stato valido.

Esempio di lista di inizializzazione

```
counter() : value_{0} {}
```

18.2 Ordine di inizializzazione

I membri vengono inizializzati **nell'ordine in cui sono dichiarati nella classe**, non nell'ordine in cui compaiono nella lista di inizializzazione. Questo comportamento è fondamentale per evitare errori sottili.

Regola importante

L'ordine di inizializzazione segue l'ordine delle dichiarazioni nella classe, non l'ordine nella lista.

Questo significa che, se i membri hanno dipendenze tra loro, è necessario dichiararli nell'ordine corretto.

18.3 Quando la lista di inizializzazione è obbligatoria

Per alcuni tipi di membri, la lista di inizializzazione non è solo consigliata, ma necessaria. In particolare:

- **riferimenti** (`T&`) Devono essere inizializzati al momento della costruzione; non possono essere riassegnati.
- **membri `const`** Devono ricevere un valore iniziale al momento della costruzione.
- **membri che non hanno un costruttore di default** Devono essere inizializzati esplicitamente, altrimenti il compilatore non saprà come costruirli.

Esempi di membri che richiedono inizializzazione

```cpp
class example {
public:
    example(int& ref, int x)
        : ref_{ref},            // obbligatorio
          constant_{x},         // obbligatorio
          member_{x} {}         // obbligatorio se member_ non ha default ctor
private:
    int& ref_;
    const int constant_;
    SomeType member_;
};
```

18.4 Vantaggi progettuali

L'uso sistematico delle liste di inizializzazione porta benefici significativi:

- garantisce che gli invarianti siano stabiliti immediatamente;
- evita inizializzazioni doppie (default + assegnazione);
- migliora le prestazioni;
- rende il codice più chiaro e idiomatico;
- permette di inizializzare correttamente tipi complessi o non assegnabili.

Osservazione 19. *La lista di inizializzazione è uno degli strumenti più potenti del C++ per costruire oggetti robusti e coerenti. Usarla correttamente è un segno distintivo di codice professionale.*

19 Liste di Inizializzazione dei Membri

La lista di inizializzazione del costruttore è uno degli strumenti più importanti del C++ moderno per garantire che gli oggetti vengano costruiti in modo corretto, efficiente e coerente con gli invarianti della classe. A differenza delle assegnazioni nel corpo del costruttore, la lista di inizializzazione permette di inizializzare i membri *prima* dell'esecuzione del corpo stesso, assicurando che ogni membro nasca già nello stato desiderato.

19.1 Inizializzazione prima del corpo del costruttore

Una lista di inizializzazione ha la forma:

```
counter() :  value_{0} {}
```

e viene eseguita prima dell'ingresso nel corpo del costruttore. Questo significa che:

- i membri vengono costruiti direttamente con i valori desiderati;
- non si verifica alcuna inizializzazione predefinita seguita da assegnazione;
- l'oggetto inizia la sua vita già in uno stato valido.

Esempio di lista di inizializzazione

```
counter() : value_{0} {}
```

19.2 Ordine di inizializzazione

I membri vengono inizializzati **nell'ordine in cui sono dichiarati nella classe**, non nell'ordine in cui compaiono nella lista di inizializzazione. Questo comportamento è fondamentale per evitare errori sottili.

Regola importante

L'ordine di inizializzazione segue l'ordine delle dichiarazioni nella classe, non l'ordine nella lista.

Questo significa che, se i membri hanno dipendenze tra loro, è necessario dichiararli nell'ordine corretto.

19.3 Quando la lista di inizializzazione è obbligatoria

Per alcuni tipi di membri, la lista di inizializzazione non è solo consigliata, ma necessaria. In particolare:

- **riferimenti** (`T&`) Devono essere inizializzati al momento della costruzione; non possono essere riassegnati.
- **membri `const`** Devono ricevere un valore iniziale al momento della costruzione.
- **membri che non hanno un costruttore di default** Devono essere inizializzati esplicitamente, altrimenti il compilatore non saprà come costruirli.

Esempi di membri che richiedono inizializzazione

```
class example {
public:
    example(int& ref, int x)
        : ref_{ref},          // obbligatorio
          constant_{x},       // obbligatorio
          member_{x} {}       // obbligatorio se member_ non
             ha default ctor
private:
    int& ref_;
    const int constant_;
    SomeType member_;
};
```

19.4 Vantaggi progettuali

L'uso sistematico delle liste di inizializzazione porta benefici significativi:

- garantisce che gli invarianti siano stabiliti immediatamente;
- evita inizializzazioni doppie (default + assegnazione);
- migliora le prestazioni;
- rende il codice più chiaro e idiomatico;
- permette di inizializzare correttamente tipi complessi o non assegnabili.

Osservazione 20. *La lista di inizializzazione è uno degli strumenti più potenti del C++ per costruire oggetti robusti e coerenti. Usarla correttamente è un segno distintivo di codice professionale.*

20 Ordine di Inizializzazione

L'ordine con cui i membri dati vengono inizializzati è un aspetto cruciale della costruzione degli oggetti in C++. Anche quando si utilizza una lista di inizializzazione, il compilatore segue sempre l'ordine delle dichiarazioni all'interno della classe, indipendentemente dall'ordine scritto nella lista. Comprendere questa regola evita bug sottili e avvisi del compilatore, soprattutto quando i membri hanno dipendenze tra loro.

20.1 L'ordine delle dichiarazioni è determinante

I membri vengono inizializzati seguendo rigorosamente l'ordine in cui compaiono nella classe. Questo significa che, se la classe è dichiarata così:

Ordine di dichiarazione

```cpp
class example {
private:
    int x_;
    int y_;
public:
    example() : y_{1}, x_{2} {}   // L'ordine nella lista NON conta
};
```

l'inizializzazione avverrà comunque in questo ordine:

1. `x_` 2. `y_`

anche se nella lista di inizializzazione i valori compaiono nell'ordine inverso.

20.2 Perché l'ordine della lista non cambia nulla

La lista di inizializzazione specifica *come* inizializzare i membri, ma non *quando*. Il compilatore segue sempre la sequenza delle dichiarazioni nella classe, perché:

- garantisce coerenza e prevedibilità;
- evita dipendenze circolari o comportamenti indefiniti;
- permette al compilatore di ottimizzare la costruzione degli oggetti.

20.3 Rischi di un ordine incoerente

Scrivere la lista di inizializzazione in un ordine diverso da quello delle dichiarazioni può portare a:

- warning del compilatore;
- bug difficili da individuare quando un membro dipende da un altro già inizializzato;
- confusione per chi legge il codice.

Un esempio problematico:

Esempio di potenziale bug

```cpp
class pair_sum {
private:
    int a_;
    int sum_;
public:
    pair_sum(int x) : sum_{a_ + x}, a_{x} {}
    // sum_ viene inizializzato PRIMA di a_, quindi usa un valore non definito!
};
```

20.4 Buona pratica consigliata

Per evitare problemi e mantenere il codice leggibile:

- mantenere la lista di inizializzazione nello stesso ordine delle dichiarazioni;
- raggruppare logicamente i membri correlati;
- documentare eventuali dipendenze tra membri.

Stile raccomandato

```cpp
class pair_sum {
private:
    int a_;
    int sum_;
public:
    pair_sum(int x) : a_{x}, sum_{a_ + x} {}   // Ordine coerente e sicuro
};
```

20.5 Impatto sulla progettazione

L'ordine di inizializzazione non è un dettaglio secondario: influisce sulla correttezza, sulla leggibilità e sulla robustezza della classe. Rispettarlo sistematicamente permette di evitare errori sottili e di mantenere invarianti solidi.

21 Quando il Costruttore di Default Non Viene Generato

Il C++ genera automaticamente alcuni costruttori impliciti, tra cui il costruttore di default. Tuttavia, questa generazione automatica segue regole precise. Comprendere tali regole è essenziale per evitare errori di compilazione e per progettare classi che comunichino chiaramente quali operazioni sono permesse.

21.1 Regola fondamentale: dichiarare un costruttore sopprime quello di default

Se una classe **non dichiara alcun costruttore**, il compilatore genera automaticamente un costruttore di default. Ma se la classe **dichiara almeno un costruttore**, allora:

- il costruttore di default **non viene più generato automaticamente**;
- se serve, deve essere richiesto esplicitamente.

Regola chiave

Dichiarare qualunque costruttore sopprime la generazione implicita del costruttore di default.

Questo comportamento evita ambiguità: se il programmatore definisce un costruttore personalizzato, il compilatore assume che la costruzione dell'oggetto richieda logiche specifiche.

21.2 Richiedere esplicitamente il costruttore di default

Quando si desidera mantenere un costruttore di default accanto ad altri costruttori, è necessario dichiararlo esplicitamente usando la sintassi:

```
counter() = default;
```

Questa forma:

- richiede al compilatore di generare comunque il costruttore di default;
- rende esplicita l'intenzione progettuale;
- documenta che la costruzione "vuota" dell'oggetto è consentita e sicura.

Esempio: costruttore di default esplicitamente richiesto

```
class counter {
public:
    counter() = default;     // Richiesto esplicitamente
    counter(int start) : value_{start} {}
private:
    int value_;
};
```

21.3 Costruttori implicitamente generati: riepilogo

Il compilatore genera automaticamente un costruttore di default solo se:

- non è stato dichiarato alcun costruttore;
- tutti i membri dati sono inizializzabili automaticamente.

Se invece la classe dichiara almeno un costruttore, il costruttore di default **non** viene generato.

21.4 Costruttori esplicitamente defaulted

La sintassi `= default` può essere usata non solo per il costruttore di default, ma anche per altri costruttori speciali, come il costruttore di copia o l'operatore di assegnazione. Questo approccio:

- rende esplicita l'intenzione del progettista;
- documenta quali operazioni sono permesse;
- evita implementazioni manuali inutili.

Esempio: costruttori esplicitamente defaulted

```
class point {
public:
    point() = default;                    // Costruttore di
        default
    point(point const&) = default;        // Costruttore di copia
};
```

21.5 Impatto sulla progettazione

L'uso di costruttori esplicitamente defaulted:

- chiarisce quali operazioni sono supportate dalla classe;
- evita generazioni implicite indesiderate;
- migliora la leggibilità e la documentazione interna;
- permette di mantenere invarianti chiari e controllati.

Questa pratica è particolarmente utile in classi che gestiscono risorse, dove è importante controllare quali operazioni di copia, spostamento o costruzione sono consentite.

22 = default e = delete

Il C++ moderno offre due strumenti molto potenti per controllare in modo preciso quali operazioni una classe permette: `= default` e `= delete`. Queste due specifiche rendono esplicita l'intenzione del progettista, migliorano la leggibilità e impediscono usi impropri degli oggetti già in fase di compilazione.

22.1 Controllare cosa è permesso

Le due parole chiave hanno scopi complementari:

- `= default` Richiede al compilatore di generare l'implementazione standard di un costruttore o operatore speciale. È utile quando si vuole mantenere il comportamento predefinito, ma rendere esplicito che tale operazione è consentita.
- `= delete` Proibisce un'operazione, impedendone l'uso a compile-time. È fondamentale per classi che non devono essere copiabili, assegnabili o costruibili in certi modi.

Idea chiave

`= default` abilita esplicitamente un'operazione; `= delete` la disabilita esplicitamente.

22.2 Perché usarli

Questi strumenti migliorano la progettazione delle classi perché:

- rendono chiaro quali operazioni sono permesse o vietate;
- evitano implementazioni manuali inutili;
- impediscono errori di utilizzo già in fase di compilazione;
- documentano l'intenzione del progettista in modo immediato e leggibile.

Sono particolarmente utili in classi che gestiscono risorse (file, socket, memoria dinamica), dove la copia potrebbe essere pericolosa o inefficiente.

22.3 Esempio: classe non copiabile

Il seguente esempio mostra una classe `widget` che può essere costruita, ma non copiata né assegnata:

Codice C++: uso di `= default` e `= delete`

```cpp
class widget {
public:
    widget() = default;                          // Costruttore di default consentito
    widget(widget const&) = delete;              // Copia vietata
    widget& operator=(widget const&) = delete; // Assegnazione vietata
};
```

In questo caso:

- l'oggetto può essere creato normalmente;
- non può essere copiato;
- non può essere assegnato;
- eventuali tentativi di copia generano un errore di compilazione chiaro e immediato.

22.4 Impatto sulla progettazione

L'uso di `= default` e `= delete` permette di:

- definire classi più sicure e robuste;
- evitare comportamenti impliciti indesiderati;
- controllare con precisione la semantica di copia e movimento;
- rendere il codice più leggibile e auto-documentante.

Questa combinazione è uno dei pilastri del C++ moderno, soprattutto nello sviluppo di librerie e componenti complessi.

23 Costruttori Eliminati

I costruttori eliminati permettono di vietare esplicitamente alcune operazioni su una classe, impedendone l'uso già in fase di compilazione. Questo meccanismo è fondamentale quando si vuole preservare gli invarianti della classe o impedire comportamenti che non avrebbero senso per un certo tipo di oggetto.

23.1 Bloccare operazioni a compile time

Dichiarare un costruttore come `= delete` significa che quella particolare operazione non è permessa. Qualsiasi tentativo di utilizzarla produrrà un errore di compilazione chiaro e immediato. Questo approccio è molto più sicuro rispetto a lasciare che l'operazione venga generata implicitamente dal compilatore.

Le operazioni che possono essere eliminate includono:

- costruttore di copia;
- costruttore di movimento;
- operatore di assegnazione (copia o movimento);
- costruttori con firme indesiderate.

23.2 Rafforzare gli invarianti della classe

Vietare la copia o l'assegnazione è spesso necessario quando:

- l'oggetto gestisce risorse non copiabili (file, socket, mutex);
- la duplicazione dell'oggetto violerebbe un invariante logico;
- si vuole evitare copie costose o pericolose;
- l'oggetto rappresenta un'entità unica (pattern Singleton, handle non duplicabili, ecc.).

In questi casi, impedire la copia è parte integrante del design della classe.

23.3 Esempio: classe non copiabile

Il seguente esempio mostra una classe che può essere costruita, ma non copiata:

Codice C++: costruttore eliminato

```
class no_copy {
public:
    no_copy() = default;                    // Costruttore di
        default consentito
    no_copy(no_copy const&) = delete;       // Copia vietata
};
```

In questo caso:

- l'oggetto può essere creato normalmente;
- non può essere copiato;
- eventuali tentativi di copia generano un errore di compilazione.

23.4 Impatto sulla progettazione

L'uso di costruttori eliminati permette di:

- rendere esplicite le operazioni vietate;
- prevenire errori logici o violazioni degli invarianti;
- migliorare la sicurezza e la robustezza del codice;
- documentare chiaramente l'intenzione del progettista.

Questa tecnica è particolarmente utile nelle classi che rappresentano risorse uniche o non duplicabili, e nelle API che devono essere utilizzate in modo rigoroso.

24 Accessors

Gli *accessors* (o "getter") permettono di leggere lo stato interno di un oggetto senza esporre direttamente i membri dati. Sono una componente essenziale dell'incapsulamento: consentono l'accesso controllato alle informazioni mantenendo la protezione degli invarianti e la separazione tra interfaccia e implementazione.

24.1 Leggere lo stato senza esporlo

Un accessor è tipicamente una funzione membro `const` che restituisce il valore di un membro dati. Questo approccio offre due vantaggi fondamentali:

- lo stato interno rimane privato e protetto;
- l'oggetto può controllare come e cosa viene restituito.

Gli accessors non modificano lo stato e quindi sono marcati `const`, rendendoli utilizzabili anche su oggetti costanti.

24.2 Scelta del tipo di ritorno

La scelta tra restituire per valore o per riferimento dipende dal tipo del dato:

- **per valore** quando il tipo è piccolo e poco costoso da copiare (come `int`, `double`);
- **per riferimento costante** quando il tipo è grande o costoso da copiare (come `std::string`, `std::vector`, oggetti complessi).

Restituire un riferimento non costante è raro e va fatto solo quando si vuole permettere la modifica controllata del membro, ma ciò indebolisce l'incapsulamento.

24.3 Esempio di accessor

Accessor semplice per un tipo integrale

```
class counter {
public:
    int value() const { return value_; }   // getter: non
       modifica lo stato
private:
    int value_{0};
};
```

In questo esempio:

- `value()` è un osservatore puro;
- restituisce una copia del valore interno, ideale per un tipo leggero come `int`;
- mantiene l'incapsulamento perché `value_` rimane privato.

24.4 Ruolo degli accessors nella progettazione

Gli accessors contribuiscono a una progettazione robusta perché:

- permettono di controllare l'accesso allo stato interno;
- mantengono invarianti e coerenza;
- rendono l'interfaccia della classe chiara e leggibile;
- evitano l'esposizione diretta dei membri, che porterebbe a un design fragile.

In classi più complesse, gli accessors possono anche applicare logiche aggiuntive, come validazioni, conversioni o formattazioni, mantenendo l'implementazione nascosta all'utente.

25 Overloading degli Operatori

Il sovraccarico degli operatori permette di estendere la sintassi naturale del C++ a tipi definiti dall'utente, rendendo il codice più leggibile e più vicino alla notazione matematica o concettuale del dominio modellato. Un caso molto comune è la stampa su stream tramite l'operatore «, che consente di integrare un tipo personalizzato con gli stream standard come `std::cout`.

25.1 Stampa su stream con una funzione libera

Per supportare l'operatore « è pratica comune definire una funzione libera (non membro). Questa scelta è importante perché:

- permette la conversione implicita sul lato destro dell'operatore;
- mantiene l'incapsulamento: la classe espone solo gli accessors necessari;
- evita di rendere la funzione un membro, cosa che impedirebbe la simmetria dell'operatore.

Esempio: stampa di un `counter`

```
std::ostream& operator<<(std::ostream& out, counter const& c)
{
    return out << c.value();   // usa l'accessor, non accede
       ai membri privati
}
```

Questa funzione utilizza l'osservatore `value()` per ottenere lo stato interno senza violare l'incapsulamento.

25.2 Esempio completo: un tipo razionale

Un esempio più articolato è una classe che rappresenta un numero razionale (frazione). Questo tipo richiede invarianti specifici, come:

- il denominatore non deve essere zero;
- la frazione deve essere ridotta ai minimi termini;
- il segno deve essere gestito in modo coerente.

Per mantenere questi invarianti, la classe delega tutto il lavoro a una funzione privata `normalize()`, chiamata dai costruttori e da eventuali modificatori.

Classe `rational` con invarianti centralizzati

```
class rational {
public:
    rational(int n, int d) : num_{n}, den_{d} {
        normalize();            // garantisce invarianti
    }

    int numerator() const { return num_; }
    int denominator() const { return den_; }

private:
    void normalize();           // riduce la frazione, gestisce
        il segno
    int num_;
    int den_;
};
```

25.3 Centralizzare la logica degli invarianti

La funzione `normalize()` è il cuore della classe. Concentrando lì tutta la logica degli invarianti si ottengono diversi vantaggi:

- ogni costruttore o modificatore può semplicemente chiamarla;
- gli invarianti sono garantiti in ogni punto della vita dell'oggetto;
- il codice rimane più leggibile e meno duplicato;
- eventuali modifiche future alla logica richiedono interventi in un solo punto.

Questo approccio è un esempio eccellente di progettazione orientata agli oggetti: ogni classe mantiene la propria coerenza interna attraverso un punto unico di controllo.

25.4 Estendere l'esempio: stampa di un `rational`

Una volta definiti gli accessors, è naturale sovraccaricare l'operatore « anche per `rational`:

Stampa di un numero razionale

```
std::ostream& operator<<(std::ostream& out, rational const& r)
{
    return out << r.numerator() << '/' << r.denominator();
}
```

La funzione non accede ai membri privati, ma utilizza gli osservatori, mantenendo l'incapsulamento.

25.5 Impatto sulla progettazione

Il sovraccarico degli operatori, quando usato con criterio, migliora:

- la leggibilità del codice;
- la naturalezza dell'interfaccia;
- la coerenza con il dominio matematico o applicativo;
- la separazione tra interfaccia pubblica e implementazione interna.

L'esempio del tipo razionale mostra come combinare operatori, accessors e invarianti per ottenere una classe robusta e intuitiva.

26 Passaggio di Oggetti

Il modo in cui una funzione riceve un oggetto influisce su prestazioni, semantica e sicurezza del codice. La scelta tra passaggio per valore, per riferimento costante o per riferimento non costante dipende dal costo della copia e dall'intenzione della funzione rispetto allo stato dell'oggetto.

26.1 Valore o riferimento: criteri di scelta

Il passaggio degli oggetti segue tre linee guida fondamentali:

1. **Per valore** quando il tipo è piccolo e la copia è economica (ad esempio `int`, `double`, piccoli struct trivially copyable). La funzione riceve una copia indipendente dell'oggetto.

2. **Per riferimento costante (`const&`)** quando l'oggetto è grande o costoso da copiare. La funzione evita la copia e garantisce che l'oggetto non venga modificato.

3. **Per riferimento non costante (`&`)** quando la funzione deve modificare l'argomento. La modifica avviene direttamente sull'oggetto originale.

Queste scelte permettono di bilanciare efficienza, chiarezza semantica e rispetto degli invarianti.

26.2 Esempi di funzioni con passaggio per riferimento

Esempi di passaggio per riferimento

```
1 void print_counter(counter const& c);    // non copia, non
    modifica
2 void increment(counter& c);              // modifica l'oggetto
```

Nel primo caso, `print_counter` riceve un riferimento costante: nessuna copia e nessuna possibilità di alterare lo stato. Nel secondo caso, `increment` riceve un riferimento non costante: la funzione può modificare l'oggetto originale.

26.3 Impatto sulla progettazione

La scelta corretta del metodo di passaggio:

- evita copie inutili e migliora le prestazioni;
- chiarisce l'intenzione della funzione (osservatore vs modificatore);
- mantiene gli invarianti della classe;
- rende l'interfaccia più leggibile e idiomatica.

In classi complesse, il passaggio per `const&` è quasi sempre la scelta predefinita per funzioni che leggono lo stato, mentre il passaggio per riferimento non costante è riservato ai modificatori espliciti.

27 Checklist di Progettazione

Una buona classe in C++ nasce da una serie di decisioni consapevoli riguardo ai dati che deve contenere, alle operazioni che deve offrire e agli invarianti che deve mantenere. Una checklist strutturata aiuta a garantire che ogni classe sia coerente, sicura e facile da usare.

27.1 Quali dati servono?

La prima domanda riguarda lo stato interno dell'oggetto. I membri dati devono essere:

- privati, per proteggere l'incapsulamento;
- essenziali, evitando ridondanze;
- coerenti con gli invarianti della classe.

Stabilire i dati necessari permette di definire chiaramente la responsabilità della classe.

27.2 Quali operazioni deve offrire la classe?

L'interfaccia pubblica (API) deve essere progettata con attenzione:

- quali funzioni permettono di osservare lo stato?
- quali funzioni modificano lo stato?
- quali operazioni devono essere vietate (tramite `= delete`)?

Una buona API è minimale ma completa, e riflette l'uso previsto della classe.

27.3 Quali invarianti devono sempre essere rispettati?

Gli invarianti definiscono le condizioni che devono essere vere per ogni oggetto valido. Alcuni esempi:

- un denominatore non può essere zero;
- un contatore non può avere valori negativi;
- un puntatore interno deve essere sempre valido.

Gli invarianti guidano la progettazione dei costruttori e dei modificatori.

27.4 Come garantire oggetti validi al momento della costruzione?

La costruzione è il momento critico in cui gli invarianti devono essere stabiliti. Per questo:

- usare liste di inizializzazione;
- evitare membri non inizializzati;
- centralizzare la logica di validazione (ad esempio in `normalize()`).

Un oggetto deve nascere già valido, senza richiedere ulteriori chiamate.

27.5 Quali funzioni devono essere `const`?

La distinzione tra osservatori e modificatori è essenziale:

- le funzioni che leggono lo stato devono essere `const`;
- le funzioni che modificano lo stato non possono esserlo;
- marcare correttamente le funzioni aiuta il compilatore e documenta l'intenzione.

Questa scelta migliora la sicurezza e la leggibilità della classe.

28 Esercizio: progettare una classe `timer`

L'obiettivo è progettare una classe semplice ma completa, che rappresenti un timer in minuti e secondi, rispetti un invariante interno e supporti la stampa in formato `mm:ss`. La progettazione segue tre passi fondamentali.

28.1 Passo 1: dati necessari e invariante

- La classe deve rappresentare un tempo espresso in minuti e secondi.
- I membri dati devono essere privati:
  - `int minutes_;`
  - `int seconds_;`
- L'invariante fondamentale è:

$$0 \leq \texttt{seconds_} \leq 59$$

- I minuti devono essere sempre non negativi.

28.2 Passo 2: costruzione e avanzamento del timer

- Il costruttore deve accettare minuti e secondi e garantire l'invariante.
- La funzione privata `normalize()` centralizza la logica di validazione.
- La funzione `tick()` avanza il timer di un secondo rispettando l'invariante.

28.3 Passo 3: stampa del timer

- L'operatore « deve essere una funzione libera.
- Deve usare gli accessors `minutes()` e `seconds()`.
- La stampa deve essere formattata con due cifre per minuti e secondi.

28.4 Codice Finale (a blocchi per migliorare estetica ed evitare tagli)

Blocco 1: Header e dichiarazione della classe

```cpp
#include <iostream>      // per std::cout e std::ostream
#include <iomanip>       // per std::setw e std::setfill

class timer {
public:
    // Costruttore: riceve minuti e secondi
    timer(int minutes, int seconds)
        : minutes_{minutes}, seconds_{seconds} { // inizializzazione membri
        normalize();                             // garantisce l'invariante
    }

    // tick(): avanza il timer di un secondo
    void tick() {
        ++seconds_;             // aumenta i secondi
        if (seconds_ >= 60) {   // se superano 59...
            seconds_ = 0;       // ...si azzerano
            ++minutes_;         // ...e si incrementano i minuti
        }
    }

    // Osservatori: non modificano lo stato
    int minutes() const { return minutes_; }
    int seconds() const { return seconds_; }
```

Blocco 1 PT2: Header e dichiarazione della classe

```


private:
    // normalize(): mantiene seconds_ in [0, 59]
    void normalize() {
        if (seconds_ >= 60) {          // se i secondi sono troppi...
            minutes_ += seconds_ / 60; // trasferisce l'eccesso ai minuti
            seconds_ = seconds_ % 60;  // mantiene solo il resto
        }
        if (minutes_ < 0) {            // evita minuti negativi
            minutes_ = 0;
            seconds_ = 0;
        }
    }

    int minutes_{0};   // minuti (>= 0)
    int seconds_{0};   // secondi (0..59)
};
```

Blocco 2: Operatore di stampa

```

// Operatore << per stampare il timer in formato mm:ss
std::ostream& operator<<(std::ostream& out, timer const& t)
{
    out << std::setw(2) << std::setfill('0') << t.minutes() // stampa minuti a 2 cifre
        << ':'                                              // separatore
        << std::setw(2) << std::setfill('0') << t.seconds(); // stampa secondi a 2 cifre
    return out;                                             // ritorna lo stream
}
```

Blocco 3: main di esempio

```
int main()
{
    timer t{1, 75};        // 1 minuto e 75 secondi -> normalizzato a 2:15
    std::cout << t << '\n';

    t.tick();              // avanza di un secondo -> 2:16
    std::cout << t << '\n';

    return 0;
}
```

29 Perché le funzioni membro sono fondamentali

Una classe in C++ non è soltanto un contenitore di dati; è un tipo astratto che combina stato e comportamento. Le funzioni membro definiscono come l'oggetto può essere usato, quali operazioni sono valide e come vengono mantenuti gli invarianti interni. La qualità delle scelte progettuali sulle funzioni membro influisce direttamente sulla leggibilità, sulla manutenibilità e sulla correttezza del codice.

29.1 La classe come unità di comportamento

I membri dati rappresentano lo stato, ma sono le funzioni membro a dare significato a quello stato. Senza funzioni membro, una classe sarebbe poco più di una `struct` con incapsulamento. Le funzioni membro:

- definiscono le operazioni che l'oggetto può eseguire;
- proteggono gli invarianti interni;
- stabiliscono l'interfaccia pubblica che gli utenti della classe devono conoscere;
- separano l'implementazione dai dettagli interni.

Questa distinzione è essenziale per costruire tipi robusti e coerenti.

29.2 Impatto sulla qualità del codice

Le scelte relative alle funzioni membro influenzano diversi aspetti del progetto:

- **Leggibilità**: un'interfaccia ben progettata rende chiaro come usare la classe e quali operazioni sono permesse.
- **Manutenibilità**: funzioni membro ben organizzate riducono la duplicazione e centralizzano la logica.
- **Correttezza**: gli invarianti vengono mantenuti automaticamente se le funzioni membro sono progettate per farlo.

Una classe con funzioni membro ben definite è più facile da estendere e meno soggetta a errori.

29.3 Decidere cosa appartiene alla classe

Una delle decisioni più importanti nella progettazione è stabilire quali funzioni devono essere membri e quali devono restare esterne. Alcuni criteri utili:

- Una funzione appartiene alla classe se deve accedere allo stato privato o mantenere invarianti.
- Una funzione dovrebbe restare esterna se:
  - non richiede accesso ai dettagli interni;
  - deve supportare conversioni implicite su entrambi gli argomenti (come `operator«`);
  - rappresenta un'operazione simmetrica tra due oggetti.
- Le funzioni che modificano lo stato devono essere membri per garantire la coerenza interna.

Questa distinzione contribuisce a mantenere l'interfaccia pulita e coerente.

29.4 Effetti sulla progettazione complessiva

Le funzioni membro non sono solo strumenti operativi: definiscono la semantica del tipo. Una classe ben progettata:

- espone solo le operazioni necessarie;
- mantiene invarianti senza richiedere interventi esterni;
- separa chiaramente osservatori e modificatori;
- colloca all'esterno le funzioni che non devono conoscere i dettagli interni.

Questi principi rendono il codice più robusto e più facile da usare correttamente.

30 Funzioni membro e funzioni non–membro

Quando si progetta una classe in C++, una delle decisioni più importanti riguarda la collocazione delle funzioni: devono essere funzioni membro oppure funzioni non–membro? Questa scelta influenza direttamente la chiarezza dell'interfaccia, la protezione degli invarianti e la flessibilità del tipo. Una classe ben progettata non contiene "tutto dentro", ma distribuisce responsabilità in modo coerente.

30.1 Due categorie di funzioni

Ogni funzione che interagisce con un oggetto può appartenere a una delle due categorie:

- **Funzione membro**: fa parte dell'interfaccia della classe, accede direttamente allo stato privato e contribuisce alla semantica del tipo.
- **Funzione non–membro**: opera sugli oggetti dall'esterno, usando solo l'interfaccia pubblica; non ha accesso diretto ai membri privati.

Questa distinzione non è solo sintattica: è una scelta di design che determina come il tipo verrà usato e quanto sarà estensibile.

30.2 Quando una funzione deve essere membro

Una funzione appartiene alla classe quando:

- deve accedere allo stato privato per mantenere invarianti;
- modifica lo stato interno dell'oggetto;
- rappresenta un'operazione fondamentale per la semantica del tipo;

- deve essere virtuale o parte di un'interfaccia polimorfica;
- deve essere chiamata in modo naturale come `oggetto.operazione()`.

Esempi tipici includono:

- costruttori e distruttori;
- modificatori come `push()`, `insert()`, `tick()`;
- osservatori che leggono lo stato interno;
- operatori che modificano l'oggetto, come `operator+=`.

30.3 Quando una funzione deve restare non–membro

Una funzione dovrebbe restare esterna alla classe quando:

- non richiede accesso diretto ai membri privati;
- deve essere simmetrica rispetto agli argomenti (es. operatori aritmetici);
- deve supportare conversioni implicite su entrambi gli operandi;
- appartiene logicamente al "contesto" del tipo, non al tipo stesso;
- migliora l'incapsulamento evitando di esporre dettagli interni.

Esempi classici:

- `operator«` per la stampa su stream;
- funzioni matematiche come `abs()`, `gcd()`, `distance()`;
- operatori binari simmetrici come `operator+`.

30.4 Effetti sulla progettazione complessiva

La scelta tra membro e non–membro ha conseguenze importanti:

- **Leggibilità**: un'interfaccia pulita rende la classe più facile da usare.
- **Manutenibilità**: funzioni non–membro riducono la dipendenza dai dettagli interni.
- **Correttezza**: funzioni membro proteggono gli invarianti quando necessario.
- **Estensibilità**: funzioni non–membro permettono di aggiungere operazioni senza modificare la classe.

Una buona regola pratica è: *rendere membro solo ciò che deve essere membro.* Tutto il resto può vivere felicemente come funzione libera, mantenendo la classe più snella e più facile da evolvere.

31 Definire le funzioni membro dentro o fuori dalla classe

In C++ una funzione membro può essere definita direttamente all'interno del corpo della classe oppure all'esterno, dopo la dichiarazione. Questa scelta non è solo sintattica: separare dichiarazione e definizione permette di distinguere chiaramente tra **interfaccia** e **implementazione**, migliorando la leggibilità e la struttura del codice, soprattutto nei progetti più grandi.

31.1 Interfaccia e implementazione

La dichiarazione all'interno della classe rappresenta l'interfaccia pubblica: ciò che gli utenti del tipo devono conoscere per usare l'oggetto. La definizione esterna contiene invece i dettagli operativi, che possono essere modificati senza alterare l'interfaccia.

- **Interfaccia**: cosa fa la funzione.
- **Implementazione**: come lo fa.

Questa separazione è analoga a quella tra file `.hpp` e `.cpp`.

31.2 Definizione interna alla classe

Definire una funzione direttamente nel corpo della classe significa incorporare sia dichiarazione sia implementazione nello stesso punto. Questo approccio è tipico per:

- funzioni molto brevi (es. accessors);
- funzioni che devono essere `inline`;
- classi piccole o di uso locale.

Esempio:

```
class counter {
public:
    int value() const { return value_; } // definizione interna
private:
    int value_{0};
};
```

31.3 Definizione esterna alla classe

Le funzioni membro possono essere dichiarate nella classe e definite successivamente, usando l'operatore di risoluzione dello scope `::`. Questo approccio è preferibile quando:

- la funzione è lunga o complessa;
- si vuole mantenere l'interfaccia pulita e leggibile;
- si separano file header e file sorgente.

Esempio:

```
class counter {
public:
    int value() const;   // dichiarazione
private:
    int value_{0};
};

// definizione esterna
int counter::value() const {
    return value_;
}
```

31.4 Vantaggi della separazione

Separare dichiarazione e definizione porta diversi benefici:

- l'interfaccia rimane compatta e più facile da consultare;
- l'implementazione può evolvere senza modificare l'header;
- si riducono le dipendenze tra file;
- si migliora la modularità del progetto.

Questa distinzione diventa essenziale quando si lavora con librerie, moduli complessi o team di sviluppo.

32 Dichiarazione e definizione delle funzioni membro

In C++ è fondamentale distinguere tra ciò che *dichiara* l'interfaccia di una classe e ciò che *definisce* il comportamento effettivo. Questa separazione è alla base dell'organizzazione dei programmi in file header (`.hpp`) e file sorgente (`.cpp`), e permette di mantenere il codice leggibile, modulare e facilmente manutenibile.

32.1 Il ruolo della dichiarazione

Una dichiarazione comunica al compilatore che un'entità esiste: una funzione, un costruttore, un membro dati. Non contiene logica eseguibile, ma descrive la forma dell'interfaccia.

Nelle classi, le dichiarazioni:

- definiscono quali operazioni sono disponibili;
- non specificano come tali operazioni funzionano;
- costituiscono l'interfaccia pubblica del tipo.

Dichiarazioni: solo interfaccia

```cpp
// dichiarazione della classe: solo interfaccia
class counter {
public:
    counter();            // dichiarazione del costruttore
    void increment();     // dichiarazione di una funzione membro
    int value() const;    // dichiarazione di un osservatore
private:
    int value_;           // membro dati
};
```

In questa fase il compilatore sa che le funzioni esistono, ma non sa ancora cosa fanno.

32.2 Il ruolo della definizione

Una definizione fornisce l'implementazione concreta: il corpo della funzione, il codice eseguibile. Ogni dichiarazione deve avere esattamente una definizione (a meno che non sia `= delete` o `= default`).

Le definizioni delle funzioni membro si trovano tipicamente in un file `.cpp` e usano l'operatore di risoluzione dello scope `::`.

Definizioni: comportamento reale

```
// definizione del costruttore
counter::counter() : value_{0} {}   // inizializza value_ a 0

// definizione della funzione increment()
void counter::increment() {
    ++value_;                       // aumenta il valore
       interno
}

// definizione dell'osservatore value()
int counter::value() const {
    return value_;                  // restituisce lo stato
       interno
}
```

Qui il compilatore apprende come funziona ogni operazione.

32.3 Interfaccia vs implementazione

La distinzione tra dichiarazione e definizione permette di separare:

- **interfaccia**: ciò che l'utente della classe deve conoscere;
- **implementazione**: dettagli che possono cambiare senza modificare l'interfaccia.

Questa separazione:

- migliora la leggibilità dei file header;
- riduce le dipendenze tra moduli;
- permette di modificare l'implementazione senza toccare il codice che usa la classe;
- facilita la compilazione incrementale.

32.4 Costruttori: dichiarazione e definizione separate

Anche i costruttori seguono le stesse regole: possono essere dichiarati nella classe e definiti altrove. Questo è utile quando il costruttore contiene logica complessa o inizializzazioni articolate.

33 Le funzioni membro operano sugli oggetti

Una funzione membro non è mai "astratta": ogni volta che viene chiamata, agisce su un oggetto specifico. Questo è uno dei principi fondamentali della programmazione orientata agli oggetti in C++. L'oggetto fornisce lo stato, mentre la funzione membro fornisce il comportamento che modifica o osserva quello stato.

33.1 Ogni chiamata riguarda un oggetto preciso

Quando si invoca una funzione membro, si sta chiedendo a *quel* particolare oggetto di eseguire un'azione. L'oggetto diventa il contesto implicito della chiamata, e la funzione opera sui suoi dati interni.

```
counter c;       // crea un oggetto counter
c.increment();   // increment() agisce su c
```

In questo esempio, `increment()` modifica lo stato interno dell'oggetto `c`. Se esistono più oggetti della stessa classe, ciascuno mantiene il proprio stato indipendente.

33.2 Lo stesso comportamento, stati diversi

Una singola funzione membro può essere applicata a molti oggetti diversi, ma il risultato dipende dallo stato di ciascun oggetto. Questo è ciò che rende le classi potenti: lo stesso codice implementa un comportamento comune, ma ogni istanza lo applica al proprio stato.

```
counter a;    // valore iniziale 0
counter b;    // valore iniziale 0

a.increment();   // ora a vale 1
b.increment();   // ora b vale 1, indipendentemente da a
```

Ogni oggetto possiede la propria copia dei membri dati, quindi le operazioni non interferiscono tra loro.

33.3 Implicazioni progettuali

Il fatto che le funzioni membro operino su un oggetto specifico ha conseguenze importanti:

- **incapsulamento**: solo le funzioni membro possono accedere direttamente ai dati privati;
- **coerenza**: gli invarianti della classe vengono mantenuti automaticamente attraverso le funzioni membro;
- **chiarezza**: la sintassi `oggetto.funzione()` rende evidente chi sta eseguendo l'azione;
- **riusabilità**: lo stesso comportamento è condiviso da tutte le istanze, ma applicato a stati diversi.

Questa relazione tra oggetto e funzione membro è il cuore del modello a oggetti: ogni istanza è responsabile del proprio comportamento e della propria coerenza interna.

34 Il puntatore implicito `this`

Ogni funzione membro in C++ opera sempre su un oggetto specifico. Per rendere possibile questo comportamento, il linguaggio introduce automaticamente un puntatore implicito chiamato `this`. Comprendere `this` è essenziale per capire come le funzioni membro accedono allo stato interno dell'oggetto e come il compilatore interpreta realmente il codice.

34.1 Che cos'è `this`

All'interno di ogni funzione membro, il compilatore inserisce un puntatore nascosto:

> `this` è un puntatore all'oggetto su cui la funzione è stata chiamata.

Questo significa che:

- `this` contiene l'indirizzo dell'oggetto corrente;
- ogni accesso ai membri dati è in realtà un accesso a `this->membro`;
- ogni funzione membro è concettualmente una funzione che riceve `this` come parametro implicito.

34.2 Esempio: chiamata a una funzione membro

Consideriamo una classe semplice:

```
void counter::increment() {
    ++value_;
}
```

Quando scriviamo `value_`, sembra che stiamo accedendo direttamente al membro, ma il compilatore lo interpreta così:

```
void counter::increment() {
    ++this->value_;
}
```

Ogni accesso ai membri è quindi un accesso ai membri dell'oggetto puntato da `this`.

34.3 Come avviene la chiamata

Quando scriviamo:

```
counter c;
c.increment();
```

il compilatore traduce la chiamata in qualcosa di concettualmente equivalente a:

```
increment(&c);    // this == &c
```

Naturalmente la sintassi reale è diversa, ma il modello mentale è corretto: `this` è un parametro implicito passato alla funzione.

34.4 Perché `this` è importante

Il puntatore `this` permette:

- l'accesso ai membri privati dell'oggetto;
- la distinzione tra membri e parametri con lo stesso nome;
- la restituzione dell'oggetto stesso (pattern `return *this;`);
- la costruzione di operatori come `operator+=`;
- la scrittura di metodi fluenti o concatenati.

34.5 Un esempio più completo

```
class counter {
public:
    counter(int v) : value_{v} {}

    void increment() {
        ++this->value_;       // accesso esplicito tramite this
    }

    counter& reset() {
        this->value_ = 0;     // modifica dell'oggetto corrente
        return *this;         // restituisce l'oggetto stesso
    }

private:
    int value_;
};
```

Qui `this` permette sia l'accesso ai membri sia la restituzione dell'oggetto corrente, utile per chiamate concatenate come:

```
c.reset().increment();
```

35 Funzioni membro e stato dell'oggetto

Ogni classe distingue tra operazioni che *modificano* lo stato interno e operazioni che *lo osservano*. Questa distinzione è fondamentale per la progettazione corretta delle interfacce e per permettere al compilatore di verificare che una funzione non alteri accidentalmente l'oggetto su cui opera.

35.1 Due categorie di funzioni membro

Le funzioni membro si dividono naturalmente in due gruppi:

- **Modificatori**: cambiano lo stato interno dell'oggetto.
- **Osservatori**: leggono lo stato senza modificarlo.

Un esempio semplice chiarisce la differenza:

```
void increment();      // modifica l'oggetto
int value() const;     // osserva l'oggetto
```

La funzione `increment()` aumenta il valore interno, mentre `value()` restituisce lo stato senza alterarlo.

35.2 Perché serve distinguere tra osservatori e modificatori

Il compilatore deve poter sapere se una funzione:

- può cambiare i membri privati;
- può essere chiamata su oggetti costanti;
- mantiene invarianti e coerenza interna.

Senza un meccanismo per dichiarare che una funzione non modifica l'oggetto, il compilatore non potrebbe garantire la sicurezza semantica delle operazioni sugli oggetti `const`.

35.3 Il ruolo delle funzioni membro `const`

La parola chiave `const` applicata a una funzione membro indica che la funzione:

- non modifica lo stato dell'oggetto;
- può essere chiamata su oggetti dichiarati `const`;
- permette al compilatore di rilevare errori di modifica involontaria.

Formalmente, una funzione membro `const` riceve un puntatore implicito `this` di tipo:

```
counter const* this
```

e quindi non può modificare i membri dati (a meno che non siano dichiarati `mutable`).

35.4 Esempio completo

```
class counter {
public:
    void increment() {       // modificatore
        ++value_;
    }

    int value() const {      // osservatore
        return value_;
    }

private:
    int value_{0};
};
```

In questo esempio:

- `increment()` modifica `value_`;
- `value()` è garantita come funzione di sola lettura.

Questa distinzione permette di scrivere codice più sicuro e più chiaro, e consente di usare oggetti `const` in modo corretto e prevedibile.

36 Un vero problema con `const`: funzioni che osservano lo stato

Molte funzioni non modificano gli oggetti su cui operano: si limitano a leggerne lo stato, a stamparlo o a usarlo per calcoli. In questi casi è naturale passare gli oggetti come `const&`, così da garantire che non vengano alterati e da permettere l'uso di oggetti costanti. Questa scelta, però, può rivelare immediatamente se una classe è stata progettata correttamente dal punto di vista dei qualificatori `const`.

36.1 Funzioni che osservano senza modificare

Una funzione che stampa una tabella, calcola una statistica o mostra informazioni non ha motivo di modificare gli oggetti che riceve. Per questo motivo, la firma corretta è:

```
void print_table(std::vector<record> const& records);
```

Il parametro è passato per riferimento costante, così da:

- evitare copie inutili;
- impedire modifiche accidentali;
- permettere la chiamata con oggetti `const`.

36.2 Quando emergono problemi

Se `print_table()` chiama funzioni membro degli oggetti contenuti nel vettore, allora quelle funzioni devono essere marcate come `const`. In caso contrario, il compilatore segnala un errore: una funzione non–`const` non può essere chiamata su un oggetto `const`.

Questo è un segnale prezioso: indica che la classe non distingue correttamente tra osservatori e modificatori.

36.3 Esempio del problema

Supponiamo che `record` abbia un osservatore non marcato `const`:

```
class record {
public:
    std::string name();    // <-- manca const!
private:
    std::string name_;
};
```

E poi:

```
void print_table(std::vector<record> const& records) {
    for (auto const& r : records) {
        std::cout << r.name();   // ERRORE: name() non e' const
    }
}
```

Il compilatore rifiuta la chiamata perché `r` è costante, e una funzione non–`const` potrebbe modificarlo.

36.4 La soluzione: qualificare correttamente gli osservatori

La funzione deve essere dichiarata come osservatore:

```
class record {
public:
    std::string name() const;   // osservatore corretto
private:
    std::string name_;
};
```

Ora `print_table()` può essere compilata senza problemi.

36.5 Perché questo è importante

L'uso sistematico di `const` permette di:

- documentare chiaramente quali funzioni modificano lo stato;
- evitare modifiche involontarie;
- rendere il codice più sicuro e prevedibile;
- permettere l'uso di oggetti costanti in contesti di sola lettura;
- far emergere errori di progettazione prima che diventino bug.

Il qualificatore `const` non è un dettaglio sintattico, ma uno strumento di progettazione che rafforza la semantica della classe e la sua affidabilità.

37 Perché `print()` deve essere `const`

Una funzione che stampa un oggetto non modifica il suo stato interno. Per questo motivo, quando un oggetto viene passato tramite riferimento costante (`const&`), il compilatore lo tratta come un oggetto immutabile e permette di chiamare solo funzioni membro marcate come `const`. Questa regola è fondamentale per garantire la coerenza semantica delle classi e per evitare modifiche involontarie.

37.1 Accesso tramite riferimento costante

Quando un oggetto viene passato come:

```
void print(std::ostream& out, record const& r);
```

il parametro `r` è considerato costante. Di conseguenza:

- non può essere modificato all'interno della funzione;
- possono essere chiamate solo funzioni membro dichiarate `const`;
- ogni tentativo di chiamare una funzione non–`const` produce un errore di compilazione.

37.2 La firma corretta di una funzione di stampa

Una funzione membro che stampa un oggetto non deve modificarlo. La firma corretta è quindi:

```
struct record {
    void print(std::ostream& out) const;
};
```

Il qualificatore `const` indica che:

- la funzione non cambia lo stato dell'oggetto;
- può essere chiamata su oggetti `const`;
- rispetta il contratto semantico: "questa operazione è di sola lettura".

37.3 Perché il qualificatore `const` è indispensabile

Se `print()` non fosse `const`, non sarebbe possibile chiamarla su un oggetto passato come `const&`. Questo impedirebbe l'uso naturale della funzione in contesti di sola lettura, come:

```
void print_table(std::vector<record> const& records) {
    for (auto const& r : records) {
        r.print(std::cout);   // richiede print() const
    }
}
```

Senza il qualificatore `const`, il compilatore segnalerebbe un errore, perché una funzione non–`const` potrebbe modificare l'oggetto.

37.4 Effetti sulla progettazione

Rendere `print()` una funzione `const` è una scelta progettuale corretta perché:

- documenta chiaramente che la funzione non modifica l'oggetto;
- permette l'uso della funzione in contesti di sola lettura;
- aiuta il compilatore a rilevare errori di modifica involontaria;
- rafforza la distinzione tra osservatori e modificatori.

L'uso coerente di `const` è uno dei pilastri della progettazione di classi robuste e affidabili in C++.

38 Funzioni membro `const` e stato dell'oggetto

Una classe contiene funzioni che svolgono ruoli diversi: alcune modificano lo stato interno, altre si limitano a leggerlo. Il compilatore, però, non può indovinare quali funzioni siano osservatori e quali siano modificatori; senza indicazioni esplicite, considera tutte le funzioni membro potenzialmente in grado di cambiare l'oggetto. Questo comportamento è troppo permissivo e può portare a errori difficili da individuare.

38.1 Il problema: tutte le funzioni possono modificare lo stato

Nel seguente esempio, entrambe le funzioni sono trattate come modificatrici:

```
class counter {
public:
    void increment();    // dovrebbe modificare l'oggetto
    int value();         // dovrebbe solo leggerlo
private:
    int value_;
};
```

Senza qualificatori, il compilatore non distingue tra le due intenzioni. La funzione `value()` potrebbe modificare `value_` e il compilatore non avrebbe modo di impedirlo.

38.2 La soluzione: funzioni membro `const`

Una funzione membro può dichiarare esplicitamente che non modifica l'oggetto aggiungendo il qualificatore `const` dopo la lista dei parametri. Questo crea un contratto chiaro tra la classe e chi la usa:

- la funzione non può modificare i membri dati;
- la funzione può essere chiamata su oggetti `const`;
- eventuali violazioni generano errori a tempo di compilazione.

38.3 Sintassi corretta

La versione corretta della classe è:

```
class counter {
public:
    void increment();      // modificatore
    int value() const;     // osservatore
private:
    int value_;
};
```

Il qualificatore `const` dopo `value()` indica che la funzione non può alterare lo stato dell'oggetto. Internamente, il compilatore interpreta la funzione come se ricevesse un puntatore implicito:

`counter const* this`

e quindi ogni tentativo di modificare `value_` all'interno della funzione produrrebbe un errore.

38.4 Controllo del compilatore

Il compilatore verifica rigorosamente il rispetto del contratto. Per esempio, questo codice non è valido:

```
int counter::value() const {
    ++value_;   // ERRORE: modifica non permessa
    return value_;
}
```

Il controllo avviene a tempo di compilazione, garantendo che gli osservatori rimangano tali.

38.5 Impatto sulla progettazione

L'uso coerente di funzioni membro `const` migliora la qualità del codice:

- chiarisce quali funzioni modificano lo stato e quali no;
- permette di usare oggetti `const` in modo sicuro;
- abilita l'integrazione con algoritmi e funzioni che richiedono oggetti immutabili;
- riduce la possibilità di bug dovuti a modifiche involontarie.

39 Il legame tra `const` e il puntatore implicito `this`

Ogni funzione membro riceve automaticamente un puntatore all'oggetto su cui è stata chiamata. Questo puntatore si chiama `this` ed è la chiave per capire perché una funzione membro `const` non può modificare lo stato dell'oggetto. Il qualificatore `const` non è applicato alla funzione in sé, ma all'oggetto a cui `this` punta.

39.1 Come cambia `this` nelle funzioni membro

Il tipo di `this` dipende dal fatto che la funzione membro sia `const` oppure no:

- in una funzione membro *non–const*:

  `counter* this;`

  il puntatore permette la modifica dei membri dell'oggetto;

- in una funzione membro *const*:

  `counter const* this;`

  il puntatore è a un oggetto costante, quindi i membri non possono essere modificati.

Questa differenza è ciò che impedisce a una funzione `const` di alterare lo stato interno.

39.2 Perché la modifica è vietata

Poiché `this` è un puntatore a `const` quando la funzione è dichiarata `const`, ogni accesso ai membri è trattato come:

`this->membro`

e quindi come:

`(counter const*)->membro`

Il compilatore rifiuta qualsiasi tentativo di scrivere:

```
value_ = 10;   // ERRORE in una funzione const
```

perché equivarrebbe a modificare un oggetto costante.

39.3 Esempio illustrativo

```
class counter {
public:
    void increment() {       // this: counter*
        ++value_;            // modifica permessa
    }

    int value() const {      // this: counter const*
        return value_;       // sola lettura
    }

private:
    int value_{0};
};
```

La funzione `increment()` può modificare `value_`, mentre `value()` non può farlo perché il tipo di `this` la vincola alla sola lettura.

39.4 Implicazioni progettuali

Il comportamento di `this` nelle funzioni `const` ha conseguenze importanti:

- garantisce che gli osservatori non modifichino lo stato;
- permette di usare oggetti `const` in modo sicuro;
- abilita ottimizzazioni del compilatore basate sull'immutabilità;
- rafforza la distinzione tra funzioni di lettura e funzioni di modifica.

Questa regola è uno dei pilastri della semantica delle classi in C++ e rende il comportamento degli oggetti più prevedibile e robusto.

40 Funzioni `const` e non-`const`: un esempio con `point`

La distinzione tra funzioni membro `const` e non-`const` diventa molto chiara quando si osserva una classe semplice come `point`. Alcune funzioni leggono soltanto lo stato dell'oggetto, mentre altre lo modificano. Il compilatore deve poter distinguere questi due ruoli, e il qualificatore `const` è lo strumento che permette di farlo in modo rigoroso.

40.1 Esempio di classe con osservatori e modificatori

```
struct point
{
    double distance() const { return std::sqrt(x*x + y*y); }
    double angle() const    { return std::atan2(y, x); }

    void offset(double xo, double yo) { x += xo; y += yo; }
    void scale(double xs, double ys)  { x *= xs; y *= ys; }

    double x;
    double y;
};
```

Le funzioni `distance()` e `angle()` sono osservatori: leggono lo stato senza modificarlo. Le funzioni `offset()` e `scale()` sono modificatori: cambiano i valori di `x` e `y`.

40.2 Regola fondamentale: cosa può chiamare un oggetto `const`

Il comportamento del compilatore segue una regola semplice e molto importante:

- un oggetto **non-`const`** può chiamare qualsiasi funzione membro;
- un oggetto `const` può chiamare solo funzioni membro dichiarate `const`.

La situazione più comune in cui un oggetto diventa `const` è quando viene passato come riferimento costante:

```
void print_polar(point const& pt)
{
    std::cout << pt.distance() << ' '
              << pt.angle()    << '\n';
}
```

Qui `pt` è un oggetto costante. Per questo motivo:

- `distance()` deve essere `const`;
- `angle()` deve essere `const`;
- funzioni come `offset()` e `scale()` non possono essere chiamate.

Se `distance()` o `angle()` non fossero `const`, il compilatore rifiuterebbe la chiamata, perché una funzione non-`const` potrebbe modificare l'oggetto.

40.3 Perché questa regola è essenziale

La distinzione tra funzioni `const` e non-`const` permette di:

- garantire che gli osservatori non modifichino lo stato;
- usare oggetti `const` in contesti di sola lettura;
- rilevare errori di progettazione a tempo di compilazione;
- rendere il codice più sicuro e più prevedibile.

L'uso corretto di `const` è uno dei pilastri della progettazione di classi robuste in C++.

Se vuoi approfondire, posso mostrarti come la stessa distinzione si applica agli operatori, ad esempio `operator[]` o `operator+`, che spesso richiedono versioni sia `const` sia non-`const`.

41 Livelli di accesso nelle classi

Il modello a oggetti di C++ permette di controllare in modo preciso quali parti di una classe sono visibili all'esterno e quali devono rimanere nascoste. Questo meccanismo è fondamentale per proteggere la rappresentazione interna e per definire un'interfaccia chiara e stabile. I livelli di accesso (`public`, `private`, `protected`) sono gli strumenti che consentono questa separazione.

41.1 Interfaccia pubblica e dettagli privati

Una classe ben progettata espone solo ciò che è necessario per usarla e nasconde tutto ciò che riguarda l'implementazione interna. Questa distinzione migliora l'incapsulamento, riduce la possibilità di errori e permette di modificare l'implementazione senza rompere il codice che utilizza la classe.

- **Membri pubblici**: costituiscono l'interfaccia della classe. Sono accessibili da qualsiasi parte del programma.
- **Membri privati**: rappresentano la struttura interna e gli invarianti. Sono accessibili solo dall'interno della classe.

41.2 Come si specifica il livello di accesso

Un livello di accesso è introdotto da una parola chiave seguita da due punti:

```
class counter {
public:
    void increment();    // parte dell'interfaccia
private:
    int value_;          // rappresentazione interna
};
```

Il livello di accesso rimane attivo per tutti i membri che seguono, fino a quando non viene specificato un nuovo livello.

41.3 Più livelli di accesso nella stessa classe

È possibile alternare più volte i livelli di accesso all'interno della stessa definizione di classe. Questo permette di organizzare i membri in modo logico e leggibile.

```
class example {
public:
    void f();        // accessibile dall'esterno

private:
    int data_;       // nascosto

public:
    void g();        // di nuovo pubblico
};
```

Questa flessibilità è utile quando si vuole raggruppare funzioni correlate o separare sezioni dell'interfaccia.

41.4 Perché i livelli di accesso sono importanti

L'uso corretto degli access specifiers ha un impatto diretto sulla qualità del design:

- protegge gli invarianti interni impedendo modifiche non controllate;
- rende l'interfaccia più chiara e più facile da usare;
- riduce la dipendenza dai dettagli di implementazione;
- permette di evolvere la classe senza rompere il codice esistente.

L'incapsulamento è uno dei principi fondamentali della programmazione orientata agli oggetti, e i livelli di accesso sono lo strumento che lo rende possibile in C++.

42 Membri `public` e `private`

Il sistema dei livelli di accesso in C++ stabilisce quali parti di una classe sono visibili dall'esterno e quali devono rimanere protette. Questa distinzione è uno dei cardini dell'incapsulamento e permette di controllare come un oggetto può essere usato, evitando modifiche non autorizzate alla sua rappresentazione interna.

42.1 Cosa è consentito ai membri `public`

I membri dichiarati `public` sono accessibili da qualsiasi codice che possa vedere l'oggetto. Questo significa che:

- funzioni esterne possono chiamare funzioni membro pubbliche;
- il codice client può creare oggetti, invocare metodi e leggere eventuali membri pubblici;
- l'interfaccia pubblica definisce tutto ciò che è necessario per usare correttamente la classe.

In pratica, la sezione `public` rappresenta il contratto tra la classe e il mondo esterno.

42.2 Cosa è consentito ai membri `private`

I membri dichiarati `private` sono accessibili solo alle funzioni membro della classe stessa. Nessun codice esterno può leggerli o modificarli direttamente. Questo permette di:

- proteggere la rappresentazione interna;
- garantire che gli invarianti siano mantenuti;
- evitare dipendenze indesiderate dai dettagli di implementazione;
- modificare liberamente l'implementazione senza rompere il codice client.

La sezione `private` è quindi il cuore nascosto dell'oggetto, dove risiede lo stato e dove si preserva la coerenza interna.

42.3 Il ruolo di `protected`

Esiste anche il livello `protected`, che diventa rilevante quando si introduce l'ereditarietà. I membri `protected` sono accessibili:

- alla classe stessa;
- alle classi derivate;
- ma non al codice esterno.

Questo livello intermedio permette alle sottoclassi di estendere il comportamento mantenendo comunque un certo grado di incapsulamento.

42.4 Implicazioni progettuali

La scelta tra `public`, `private` e `protected` ha un impatto diretto sulla qualità del design:

- un'interfaccia pubblica ben definita rende la classe facile da usare e difficile da usare male;
- i membri privati proteggono la struttura interna e riducono la fragilità del codice;
- l'uso ponderato di `protected` facilita l'estensibilità tramite ereditarietà.

Un buon progetto tende a minimizzare ciò che è pubblico e a massimizzare ciò che è privato, esponendo solo ciò che è realmente necessario.

43 Implementazione della classe `point`: interfaccia pubblica e rappresentazione privata

La versione completa della classe `point` mostra in modo molto chiaro come separare l'interfaccia pubblica dalla rappresentazione interna. Le funzioni pubbliche definiscono ciò che un oggetto può fare; i dati privati definiscono come l'oggetto è rappresentato. Questa distinzione è fondamentale per proteggere gli invarianti e mantenere un design robusto.

43.1 Interfaccia pubblica: costruttori e osservatori

```
struct point
{
public:
    point() : point{0.0, 0.0} {}
    point(double x, double y) : x_{x}, y_{y} {}
    point(point const&) = default;

    double x() const { return x_; }
    double y() const { return y_; }

    double angle() const
    { return std::atan2(y(), x()); }

    double distance() const
    { return std::sqrt(x()*x() + y()*y()); }
```

Questa parte della classe definisce:

- **costruttori** per creare punti in modi diversi;
- **osservatori** (`x()`, `y()`, `angle()`, `distance()`) che leggono lo stato senza modificarlo;
- un'interfaccia pulita e stabile per chi usa la classe.

Tutte queste funzioni sono `const` perché non modificano lo stato dell'oggetto.

43.2 Modificatori e rappresentazione privata

```
public:
    void move_cartesian(double x, double y)
    { x_ = x; y_ = y; }

    void move_polar(double r, double angle)
    { move_cartesian(r * std::cos(angle),
                     r * std::sin(angle)); }

    void offset(double xo, double yo)
    { move_cartesian(x() + xo, y() + yo); }

private:
    double x_;
    double y_;
};
```

Qui troviamo:

- **modificatori** che cambiano lo stato dell'oggetto;
- una rappresentazione interna completamente nascosta (`x_`, `y_`).

La scelta di rendere privati i membri dati è essenziale per mantenere il controllo sulla coerenza interna.

43.3 Cosa è cambiato rispetto a una versione ingenua

La differenza principale è l'incapsulamento:

- **i membri dati sono privati**, quindi nessun codice esterno può modificarli direttamente;
- **solo le funzioni membro possono cambiare lo stato**;
- **gli osservatori forniscono accesso controllato alla rappresentazione**.

Questo approccio protegge gli invarianti e impedisce modifiche non autorizzate.

43.4 Errore comune: accesso diretto ai membri privati

Il compilatore impedisce qualsiasi tentativo di accedere ai membri privati dall'esterno:

```
point p{1.0, 2.0};
p.x_ = 3.0;   // ERRORE: x_ e' private
```

Questo errore è intenzionale: la rappresentazione interna non deve essere manipolata direttamente dal codice client.

43.5 Perché questa struttura è corretta

Organizzare la classe in questo modo offre diversi vantaggi:

- la rappresentazione interna può cambiare senza rompere il codice esterno;
- gli invarianti sono protetti da modifiche arbitrarie;
- l'interfaccia pubblica è chiara e stabile;
- la classe è più facile da mantenere ed estendere.

L'incapsulamento è uno dei principi fondamentali della progettazione orientata agli oggetti, e questa implementazione di `point` lo illustra perfettamente.

44 Interfaccia e implementazione: perché separarli

Una classe ben progettata distingue nettamente tra ciò che *offre* al mondo esterno e ciò che *usa internamente* per funzionare. Questa separazione tra interfaccia e implementazione è uno dei principi più solidi della progettazione orientata agli oggetti in C++, perché permette di mantenere il controllo sulla rappresentazione interna e di evolvere il codice senza rompere quello esistente.

44.1 L'interfaccia: ciò che gli utenti devono conoscere

L'interfaccia è costituita dai membri `public`. Questi elementi definiscono:

- quali operazioni un oggetto può eseguire;
- quali informazioni possono essere lette dall'esterno;
- quali garanzie la classe offre ai suoi utilizzatori.

Gli utenti di un tipo non hanno bisogno di sapere come i dati sono memorizzati, ma solo quali operazioni sono disponibili e quali effetti producono.

44.2 L'implementazione: ciò che la classe controlla

L'implementazione è costituita dai membri `private`. Qui risiedono:

- la rappresentazione interna dei dati;
- gli invarianti che la classe deve mantenere;
- i dettagli che possono cambiare senza influenzare il codice esterno.

Rendere privati i dati significa che la classe può modificare liberamente la propria struttura interna, purché l'interfaccia pubblica rimanga invariata.

44.3 Perché la separazione è utile

Questa distinzione porta vantaggi concreti:

- **stabilità del codice client**: se la rappresentazione interna cambia, il codice che usa la classe non deve essere modificato;
- **protezione degli invarianti**: solo le funzioni membro possono alterare lo stato, garantendo coerenza;
- **manutenibilità**: l'implementazione può evolvere senza impatti esterni;
- **leggibilità**: l'interfaccia pubblica è più chiara e più facile da consultare.

In altre parole, gli utenti si concentrano su *cosa* un tipo può fare, mentre la classe mantiene il controllo su *come* lo fa.

44.4 Un esempio concreto

Nel caso della classe `point` vista in precedenza:

- l'interfaccia pubblica include costruttori, osservatori e modificatori;
- la rappresentazione interna (`x_`, `y_`) è privata;
- eventuali cambiamenti alla struttura interna non richiedono modifiche al codice che usa `point`.

Questo è esattamente il tipo di flessibilità che l'incapsulamento è progettato per offrire.

45 Linee guida di progettazione: mantenere i dati privati

Una classe ben progettata espone un'interfaccia chiara e stabile, mentre protegge la propria rappresentazione interna. Rendere i dati privati e fornire operazioni pubbliche è una delle regole più importanti dell'ingegneria del software orientata agli oggetti. Questa scelta non è solo stilistica: ha conseguenze profonde sulla robustezza, la manutenibilità e l'evoluzione del codice.

45.1 Perché i dati devono essere privati

I membri dati rappresentano lo stato interno dell'oggetto. Se fossero pubblici, qualsiasi parte del programma potrebbe modificarli liberamente, rendendo impossibile garantire invarianti e coerenza. Mantenere i dati privati permette alla classe di:

- controllare come e quando lo stato cambia;
- evitare modifiche arbitrarie da parte del codice esterno;
- proteggere gli invarianti logici (ad esempio: un punto non può avere coordinate non valide, un contatore non può andare in negativo se non previsto);
- cambiare la rappresentazione interna senza rompere il codice client.

Questa protezione è essenziale per mantenere un design solido e prevedibile.

45.2 Perché le operazioni devono essere pubbliche

Le funzioni membro pubbliche costituiscono l'interfaccia della classe. Sono il modo corretto per interagire con l'oggetto, perché:

- possono **validare gli input** prima di modificare lo stato;
- possono **preservare gli invarianti**, applicando regole e controlli;
- offrono una **superficie d'uso stabile**, indipendente dai dettagli interni;
- permettono di aggiungere logica (logging, controlli, ottimizzazioni) senza cambiare il codice client.

In altre parole, l'interfaccia pubblica è un contratto: promette un certo comportamento, mentre l'implementazione interna può evolvere liberamente.

45.3 Il problema dei dati pubblici

Rendere pubblici i dati crea dipendenze rigide e fragili:

- il codice esterno diventa legato alla rappresentazione interna;
- ogni cambiamento ai membri dati richiede modifiche ovunque vengano usati;
- gli invarianti possono essere violati accidentalmente;
- la classe perde il controllo sul proprio stato.

Un esempio tipico:

```
struct point {
    double x;   // pubblico: pericoloso
    double y;
};
```

Qualsiasi codice può fare:

```
p.x = std::numeric_limits<double>::quiet_NaN();
```

e la classe non ha modo di impedirlo.

45.4 Il vantaggio di un'interfaccia stabile

Quando i dati sono privati, la classe può cambiare la propria rappresentazione senza impatti esterni. Per esempio, `point` potrebbe passare da coordinate cartesiane a polari:

```
private:
    double r_;
    double theta_;
```

e nessun codice client si romperebbe, perché l'interfaccia pubblica rimane invariata.

Questa flessibilità è uno dei motivi principali per cui l'incapsulamento è considerato un principio fondamentale dell'OOP.

45.5 Sintesi

- I dati devono essere privati per proteggere la rappresentazione e gli invarianti.
- Le operazioni devono essere pubbliche per offrire un'interfaccia sicura e stabile.
- Le funzioni membro possono validare, controllare e mantenere la coerenza interna.
- I dati pubblici creano dipendenze rigide e fragili, difficili da correggere in seguito.

46 Esercizio: riscrivere `point` usando coordinate polari

L'obiettivo è mostrare in modo didattico come la separazione tra interfaccia e implementazione renda possibile cambiare la rappresentazione interna senza rompere il codice esterno. Tutte le spiegazioni sono integrate direttamente nel codice tramite commenti riga per riga, suddivise in blocchi logici.

46.1 Quali parti devono cambiare

Cambiare la rappresentazione interna da cartesiana (x,y) a polare $(r,angle)$ implica modifiche precise:

- **osservatori** (`x()`, `y()`, `distance()`, `angle()`) perché dipendono direttamente dai dati;
- **modificatori** (`move_cartesian()`, `move_polar()`, `offset()`) perché aggiornano lo stato interno;
- **costruttori**, che devono inizializzare la nuova rappresentazione;
- **nessun cambiamento** per il codice esterno che usa solo l'interfaccia pubblica.

Questo esercizio dimostra che più codice dipende dai dati privati, più sarà fragile; più codice usa solo l'interfaccia pubblica, più sarà stabile.

47 Implementazione completa con coordinate polari

47.1 Blocco 1: interfaccia pubblica e osservatori

```
struct point
{
public:

    point() : point{0.0, 0.0} {}
    // Costruttore di default: delega al costruttore principale
    // inizializzando il punto all'origine (0,0).

    point(double x, double y)
        : r_{0.0}, angle_{0.0}
    // Inizializza r_ e angle_ a valori temporanei.
    // La conversione corretta avviene tramite move_cartesian().
    {
        move_cartesian(x, y);
        // Converte (x,y) in coordinate polari.
    }

    double distance() const { return r_; }
    // Osservatore: restituisce il raggio, cioe' la distanza dall'
       origine.

    double angle() const { return angle_; }
    // Osservatore: restituisce l'angolo in radianti.

    double x() const { return r_ * std::cos(angle_); }
    // Conversione polare -> cartesiana: x = r cos(theta).

    double y() const { return r_ * std::sin(angle_); }
    // Conversione polare -> cartesiana: y = r sin(theta).
```

47.2 Blocco 2: rappresentazione nascosta e modificatori

```
public:

    void move_cartesian(double x, double y)
    {
        r_ = std::sqrt(x*x + y*y);
        // Aggiorna il raggio in base alle coordinate cartesiane.

        angle_ = std::atan2(y, x);
        // Calcola l'angolo in modo robusto (gestisce quadranti e x
           =0).
    }

    void move_polar(double r, double angle)
    {
        r_ = r;
        // Imposta direttamente il raggio.

        angle_ = angle;
        // Imposta direttamente l'angolo.
    }

    void offset(double xo, double yo)
    {
        move_cartesian(x() + xo, y() + yo);
        // Converte il punto in cartesiano, applica lo spostamento,
        // poi riconverte in polare.
    }

private:

    double r_;
    // Rappresenta la distanza dall'origine.

    double angle_;
    // Rappresenta l'angolo in radianti.
};
```

48 Cosa dimostra questo esercizio

- La rappresentazione interna può cambiare completamente senza toccare il codice esterno.
- L'interfaccia pubblica rimane stabile e continua a funzionare.
- L'incapsulamento protegge gli invarianti e riduce la fragilità del software.
- Le funzioni che dipendono dai dati privati devono essere riscritte; quelle che usano solo l'interfaccia no.

49 Classi e `struct` in C++: differenze e linee guida

In C++ le parole chiave `class` e `struct` definiscono tipi composti con le stesse capacità: possono avere costruttori, distruttori, funzioni membro, ereditarietà, operatori sovraccaricati e incapsulamento. La differenza principale riguarda il livello di accesso predefinito e, di conseguenza, l'uso idiomatico consigliato.

49.1 Differenza fondamentale: il livello di accesso predefinito

- In una `struct`, i membri sono **public** per default.
- In una `class`, i membri sono **private** per default.

Questa differenza influenza lo stile di progettazione: una `class` incoraggia l'incapsulamento, mentre una `struct` è più adatta a contenitori di dati semplici.

49.2 Quando usare `struct`: aggregati semplici

Una `struct` è appropriata quando il tipo rappresenta un insieme di dati senza invarianti complessi e senza logica interna significativa. È tipica nei casi in cui:

- il tipo è un semplice record di dati;
- i membri possono essere esposti senza rischi;
- non è necessario proteggere la rappresentazione interna;
- il tipo è usato come contenitore passivo.

Esempi comuni includono coordinate, colori, parametri di configurazione o risultati di funzioni.

49.3 Quando usare `class`: incapsulamento e invarianti

Una `class` è preferibile quando il tipo deve controllare la propria coerenza interna. In questi casi:

- la rappresentazione deve essere privata;
- le operazioni pubbliche devono validare gli input;
- gli invarianti devono essere preservati;
- l'interfaccia deve rimanere stabile anche se l'implementazione cambia.

Questo approccio riduce la possibilità di uso scorretto e rende più sicuro il refactoring.

49.4 Perché la scelta è importante

La distinzione tra `struct` e `class` non è solo sintattica, ma riflette un'intenzione progettuale:

- una `struct` comunica che il tipo è un semplice contenitore di dati;
- una `class` comunica che il tipo ha regole, invarianti e comportamento;
- l'incapsulamento protegge la rappresentazione interna e riduce le dipendenze;
- un'interfaccia ben definita permette di evolvere il tipo senza rompere il codice esistente.

Questa distinzione diventa particolarmente utile in progetti di grandi dimensioni, dove la chiarezza delle intenzioni e la protezione degli invarianti sono essenziali.

Ti interessa vedere un esempio concreto in cui un tipo nato come `struct` deve essere trasformato in `class` perché gli invarianti crescono e la rappresentazione deve essere protetta?